

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці

ДО ЗАХИСТУ ДОПУЩЕНО

В.о. завідувача кафедри
Олександр КОВАЛЬ

«__» _____ 2026 р.

Дипломна робота

**на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інтелектуальних кібер-фізичних систем в енергетиці»
спеціальності 121 Інженерія програмного забезпечення
на тему: «ГЛИБОКЕ НАВЧАННЯ З ПІДКРІПЛЕННЯМ ДЛЯ КЕРУВАННЯ
ТОПОЛОГІЄЮ ЕНЕРГОМЕРЕЖ ПРИ РАПТОВИХ ВТРАТАХ ВУЗЛІВ
ГЕНЕРАЦІЇ »**

Виконала:

студентка IV курсу, групи ТВ-23

Бунда Ольга Миколаївна

(прізвище, ім'я, по батькові)

(підпис)

Керівник:

доцент кафедри ІПЗЕ, к.т.н Стативка Ю.І.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Рецензент:

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень із праць інших авторів
без відповідних посилань.

Студентка

(підпис)

Київ – 2026

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»**

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці
Рівень вищої освіти перший (бакалаврський)
Спеціальність 121 Інженерія програмного забезпечення
Освітньо-професійна програма «Інженерія програмного забезпечення
інтелектуальних кібер-фізичних систем в енергетиці»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри
Олександр КОВАЛЬ

«__» _____ 2026р.

ЗАВДАННЯ

на дипломну роботу студенту

Бунді Ользі Миколаївні
(прізвище, ім'я, по батькові)

1. Тема роботи: Глибоке навчання з підкріпленням для керування топологією енергомереж при раптових втратах вузлів генерації

керівник роботи Стативка Юрій Іванович, доцент, кандидат технічних наук
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «28» травня 2026р. № 1992-с

2. Строк подання студентом роботи «__» _____ 2026р.

3. Вихідні дані до роботи: мова програмування Python, середовище симуляції Grid2Op, алгоритм PPO бібліотеки Stable-Baselines3, бекенд LightSim2Grid, веб-фреймворк Streamlit, бібліотека Matplotlib, бібліотека NumPy, середовище розробки Visual Studio Code

4. Зміст (дипломної роботи) пояснювальної записки (перелік завдань, які потрібно розробити): дослідити існуючі підходи до керування енергомережами, розробити математичну модель задачі, реалізувати функцію винагороди, навчити PPO агента, провести порівняльне тестування, розробити веб-застосунок

5. Перелік ілюстративного матеріалу: UML діаграми, графіки навчання агента, графіки порівняння методів, топологія мережі, інтерфейс застосунку

6. Дата видачі завдання «31» жовтня 2025р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання дипломної роботи	Строки виконання етапів роботи	Примітка
1	Отримання завдання	31.10.2025	
2	Дослідження предметної області	01.11.2025 – 31.11.2025	
3	Дослідження існуючих рішень	01.12.2025 – 31.12.2025	
4	Постановка вимог до проєктування системи	01.01.2026 – 31.01.2026	
5	Розробка програмного продукту	01.02.2026 – 03.05.2026	
6	Тестування	04.05.2026 – 10.05.2026	
7	Захист програмного продукту	11.05.2026 – 15.05.2026	
8	Оформлення дипломної роботи	18.05.2026 – 31.05.2026	
9	Передзахист	01.06.2026 – 05.06.2026	
10	Захист	15.06.2026 – 19.06.2026	

Студент

(підпис)

Ольга Бунда

(ім'я, прізвище)

Керівник роботи

(підпис)

Юрій Стативка

(ім'я, прізвище)

РЕФЕРАТ

Структура та обсяг дипломної роботи. Робота містить 52 сторінки, 12 рисунків, 7 таблиць, 2 додатки та 18 посилань.

Метою роботи є розробка програмної системи керування топологією енергомережі IEEE 14-bus на основі алгоритму Proximal Policy Optimization, що здатна запобігати блекаутам при раптових відключеннях вузлів генерації.

Практичне значення одержаних результатів полягає у розробці інтелектуальної системи керування що демонструє ефективність глибокого навчання з підкріпленням як підходу до керування топологією енергомережі. Розроблений застосунок дозволяє симулювати аварійні сценарії, порівнювати ефективність різних методів керування та візуалізувати стан мережі в реальному часі.

Ключові слова: навчання з підкріпленням, керування енергомережею, PPO, Grid2Op, IEEE 14-bus, марківський процес прийняття рішень, функція винагороди, топологія мережі, блекаут, Streamlit.

ABSTRACT

Structure and scope of the thesis. The work contains 52 pages, 12 figures, 7 table, 2 appendices and 18 references.

The purpose of the work is to develop a software system for controlling the IEEE 14-bus power grid topology based on the Proximal Policy Optimization algorithm, which is capable of preventing blackouts during sudden shutdowns of generation nodes.

The practical value of the results obtained lies in the development of an intelligent control system that demonstrates the effectiveness of deep reinforcement learning as an approach to power grid topology control. The developed application allows simulating emergency scenarios, comparing the effectiveness of different control methods, and visualizing the network status in real time.

Keywords: reinforcement learning, power grid control, PPO, Grid2Op, IEEE 14-bus, Markov decision process, reward function, network topology, blackout, Streamlit.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ.....	8
ВСТУП.....	9
1 ПОСТАНОВКА ЗАДАЧІ КЕРУВАННЯ ТОПОЛОГІЄЮ ЕНЕРГОМЕРЕЖ	11
1.1 Актуальність задачі керування енергомережами.....	11
1.2 Формулювання задачі керування топологією	12
1.3 Марківський процес прийняття рішень як модель керування.....	13
Висновки до розділу 1.....	15
2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ.....	16
2.1 Огляд алгоритмів керування енергомережами	16
2.1.1 Порівняння алгоритмів навчання з підкріпленням.....	17
2.2 Огляд інструментів симуляції енергомереж.....	18
2.3 Порівняльний аналіз підходів	20
Висновки до розділу 2.....	21
3 ЗАСОБИ РОЗРОБКИ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	22
3.1 Середовище симуляції Grid2Op.....	22
3.1.1 Структура середовища Grid2Op.....	23
3.2 Алгоритм PPO та бібліотека Stable-Baselines3	23
3.2.1 Архітектура нейромережі PPO.....	24
3.3 Мова програмування Python та веб-фреймворк Streamlit	25
Висновки до розділу 3.....	26
4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ	28
4.1 Архітектура системи	28
4.2 Математична модель задачі.....	30
4.3 Реалізація функції винагороди.....	32
4.4 Навчання агента.....	33
4.5 Тестування системи.....	35
Висновки до розділу 4.....	36
5 РОБОТА КОРИСТУВАЧА З ПРОГРАМНОЮ СИСТЕМОЮ	38
5.1 Вимоги до середовища та встановлення.....	38
5.2 Вкладка симуляції аварій.....	38
5.3 Порівняння сценаріїв	42

5.4 Архітектура системи	44
5.5 Ручне керування	46
5.6 Розгортання та подальший розвиток системи	47
Висновки до розділу 5	47
ВИСНОВКИ.....	49
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	51

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

RL	Reinforcement Learning — навчання з підкріпленням
PPO	Proximal Policy Optimization — алгоритм оптимізації проксимальної політики
MDP	Markov Decision Process — марківський процес прийняття рішень
DRL	Deep Reinforcement Learning — глибоке навчання з підкріпленням
IEEE	Institute of Electrical and Electronics Engineers
L2RPN	Learning to Run a Power Network
MILP	Mixed Integer Linear Programming — змішане цілочисельне лінійне програмування
API	Application Programming Interface — інтерфейс програмування застосунків
MLP	Multilayer Perceptron — багатошаровий перцептрон
CPU	Central Processing Unit — центральний процесор
JSON	JavaScript Object Notation — формат обміну даними
UI	User Interface — інтерфейс користувача

ВСТУП

Надійність та стабільність енергетичних систем є одним із ключових факторів сучасної інфраструктури. Раптові відключення вузлів генерації або ліній передачі можуть спричиняти каскадні аварії та масштабні блекауты, що завдають значних економічних збитків і загрожують безпеці населення. Традиційні методи керування енергомережами, зокрема жадібні алгоритми та ручне оперативне втручання, демонструють недостатню ефективність в умовах динамічних та непередбачуваних аварійних ситуацій.

Масштабні аварії в енергосистемах підтверджують актуальність цієї проблеми. Відключення електроенергії призводять не лише до прямих економічних збитків промислових підприємств та побутових споживачів, а й до порушення роботи критичної інфраструктури — медичних закладів, транспортних систем та засобів зв'язку. Зростання частки відновлюваних джерел енергії додатково ускладнює керування мережами через нестабільність генерації, що підвищує вимоги до інтелектуальних систем керування.

Глибоке навчання з підкріпленням є перспективним підходом для вирішення задач оптимального керування в складних динамічних середовищах. На відміну від класичних алгоритмів, агент на основі навчання з підкріпленням здатний самостійно виробляти ефективні стратегії керування через взаємодію із середовищем без необхідності явного програмування правил прийняття рішень.

Останніми роками дослідження у галузі застосування навчання з підкріпленням до керування енергомережами набули значного поширення. Щорічні міжнародні змагання L2RPN, що проводяться компанією RTE France починаючи з 2019 року, стимулюють розробку практичних рішень на основі RL для реалістичних моделей енергосистем. Результати цих змагань демонструють що інтелектуальні агенти здатні виробляти нетривіальні стратегії керування які перевершують традиційні підходи у складних аварійних сценаріях.

Метою даної роботи є розробка програмної системи керування топологією енергомережі IEEE 14-bus на основі алгоритму Proximal Policy Optimization (PPO), що здатна запобігати блекаутам при раптових відключеннях вузлів генерації.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- дослідити існуючі підходи до керування топологією енергомереж;
- розробити математичну модель задачі у вигляді марківського процесу прийняття рішень;
- реалізувати власну функцію винагороди для навчання агента;
- навчити PPO агента на симуляторі Grid2Op з мережею IEEE 14-bus;
- провести порівняльне тестування з класичними алгоритмами;
- розробити інтерактивний веб-застосунок для демонстрації результатів.

Дипломна робота складається з п'яти розділів. У першому розділі сформульовано постановку задачі та математичну модель. Другий розділ містить аналіз існуючих рішень і обґрунтування вибору підходу. Третій розділ описує засоби розробки. Четвертий розділ присвячено програмній реалізації системи. П'ятий розділ описує роботу користувача з веб-застосунком.

Об'єктом дослідження є процес керування топологією енергомережі в умовах аварійних відключень.

Предметом дослідження є методи глибокого навчання з підкріпленням для оптимізації керування топологією енергомережі.

Практичне значення роботи полягає у розробці програмної системи що демонструє ефективність підходу глибокого навчання з підкріпленням як proof of concept для подальшої адаптації до реальних енергосистем. Розроблений застосунок дозволяє симулювати різні типи аварій включаючи каскадні відмови типу N-1-1, порівнювати ефективність різних методів керування та візуалізувати стан мережі в реальному часі.

1 ПОСТАНОВКА ЗАДАЧІ КЕРУВАННЯ ТОПОЛОГІЄЮ ЕНЕРГОМЕРЕЖ

У розділі розглянуто задачі керування топологією енергомережі та обґрунтуванню підходу до її вирішення. Розглядається актуальність проблеми стабільного керування енергосистемами в умовах аварійних відключень, визначається математична постановка задачі оптимального керування та обирається формалізм марківського процесу прийняття рішень як основа для застосування методів навчання з підкріпленням.

1.1 Актуальність задачі керування енергомережами

Керування топологією енергомережі є технічно складною задачею через ряд фундаментальних властивостей електричних систем. На відміну від більшості технічних об'єктів, енергомережа функціонує як єдина взаємопов'язана система — зміна стану будь-якого елемента миттєво впливає на розподіл потоків потужності по всій мережі. Ця властивість описується рівняннями поточкорозподілу [17] що є нелінійними і не допускають простих аналітичних рішень.

Додаткову складність створює комбінаторна природа простору топологічних конфігурацій. Для мережі з n лініями передачі кількість можливих конфігурацій становить 2^n . При цьому більшість конфігурацій є або фізично неприпустимими через порушення зв'язності мережі, або небезпечними через критичне перевантаження окремих елементів. Визначення оптимальної конфігурації шляхом повного перебору є обчислювально неможливим в умовах реального часу.

Ще одним чинником складності є динамічний характер навантаження. Споживання електроенергії змінюється протягом доби, тижня і року відповідно до виробничих циклів та побутових потреб. Сучасні мережі з високою часткою відновлюваних джерел додатково підсилюють цю нестабільність — генерація

сонячних та вітрових електростанцій залежить від метеорологічних умов і є важко прогнозованою на короткому горизонті планування.

Зазначені чинники зумовлюють необхідність розробки адаптивних методів керування здатних приймати рішення в реальному часі з урахуванням поточного стану мережі. Методи навчання з підкріпленням є перспективним підходом до вирішення цієї задачі оскільки дозволяють агенту самостійно виробляти стратегію керування через взаємодію із середовищем без необхідності явного програмування правил.

1.2 Формулювання задачі керування топологією

Сучасні енергетичні системи являють собою складні розподілені мережі, що складаються з генераторів, ліній передачі та споживачів електроенергії. Забезпечення стабільної роботи таких мереж є критично важливим завданням, оскільки навіть короточасні перебої в електропостачанні можуть спричинити значні економічні збитки та загрозу безпеці людей.

Одним із найнебезпечніших сценаріїв є раптова втрата вузла генерації або лінії передачі. У такому випадку навантаження перерозподіляється на суміжні елементи мережі, що може призвести до їх перевантаження і подальшого каскадного відключення — так званого блекауту. Реальні приклади таких подій включають масштабні аварії в енергосистемах різних країн, що підкреслює актуальність розробки ефективних систем керування.

Традиційні підходи до керування топологією енергомереж базуються на заздалегідь визначених правилах або жадібних алгоритмах. Жадібний алгоритм, наприклад, відключає найбільш перевантажену лінію при перевищенні допустимого порогу. Проте такий підхід є неефективним — відключення лінії перерозподіляє навантаження на інші елементи мережі, що може прискорити каскадний збій замість його запобігання. Як показали результати тестування,

жадібний алгоритм у середньому виживає лише 2 кроки в критичних сценаріях порівняно з 300 кроками PPO агента.

Задача керування топологією енергомережі формулюється як задача оптимального керування в динамічному середовищі. На кожному часовому кроці системі необхідно прийняти рішення про конфігурацію активних ліній передачі таким чином, щоб мінімізувати перевантаження і запобігти блекауту. Математично ця задача записується як (формула 1.1):

$$\min_{E' \subseteq E} \sum_{e \in E'} \max(\rho_e - 0.9, 0) \quad (1.1)$$

де E' — підмножина активних ребер мережі,

E — повна множина ліній передачі,

ρ_e — коефіцієнт завантаженості лінії e ,

0.9 — допустимий поріг завантаженості (90%).

за умови збереження зв'язності графу $G' = (V, E')$, де ρ_e — коефіцієнт завантаженості лінії e .

1.3 Марківський процес прийняття рішень як модель керування

Задача керування топологією енергомережі природно описується у термінах марківського процесу прийняття рішень. Ця математична модель широко застосовується у теорії навчання з підкріпленням для формалізації задач послідовного прийняття рішень в умовах невизначеності [6].

Марківська властивість означає що майбутній стан системи визначається виключно її поточним станом і обраною дією, без урахування попередньої історії. У контексті керування енергомережею це означає що для прийняття оптимального рішення достатньо знати поточний розподіл навантажень і топологію мережі — інформація про попередні кроки не є необхідною.

Формально MDP описується кортежем (S, A, P, R, γ) де кожен елемент відповідає конкретному аспекту задачі керування. Простір станів S містить усю інформацію необхідну агенту для прийняття рішення. Простір дій A визначає множину допустимих керуючих впливів. Функція переходів P описує динаміку середовища. Функція винагороди R кількісно оцінює якість прийнятого рішення. Коефіцієнт дисконтування γ регулює відносну важливість негайних і майбутніх винагород.

У даній роботі кожен елемент MDP має конкретну реалізацію. Простір станів представлений вектором з 114 параметрів що описують поточний стан мережі IEEE 14-bus. Простір дій містить 101 дискретний варіант керування топологією. Функція переходів реалізована симулятором Grid2Op через розв'язання рівнянь потокорозподілу. Функція винагороди реалізована модулем CustomReward. Коефіцієнт дисконтування встановлено на рівні $\gamma = 0.99$.

Застосування MDP як формальної моделі дозволяє чітко визначити структуру задачі, проте не усуває її обчислювальну складність. Складність задачі обумовлена кількома факторами. По-перше, простір можливих конфігурацій є комбінаторно великим — для мережі з 20 лініями існує 2^{20} можливих топологій. По-друге, динаміка мережі є нелінійною — зміна стану однієї лінії впливає на розподіл навантаження по всій мережі. По-третє, рішення необхідно приймати в режимі реального часу без можливості повного перебору варіантів.

У даній роботі запропоновано підхід на основі глибокого навчання з підкріпленням, що дозволяє агенту самостійно виробити ефективну стратегію керування через взаємодію із симулятором енергомережі. Для моделювання використовується стандартна тестова мережа IEEE 14-bus — загальновизнаний еталон для тестування алгоритмів керування енергосистемами [5], що містить 14 вузлів, 20 ліній передачі, 6 генераторів та 11 навантажень.

Мережа IEEE 14-bus моделюється як мультиграф $G = (V, E)$, де V — множина вузлів (підстанції, генератори, навантаження), E — множина ребер (лінії передачі електроенергії). Використання мультиграфу є необхідним оскільки між двома

підстанціями може існувати кілька паралельних ліній передачі, якими необхідно керувати незалежно.

Програмна система що розроблена в рамках даної роботи включає такі компоненти:

- модуль симуляції енергомережі на базі Grid2Op з мережею IEEE 14-bus;
- модуль навчання РРО агента з власною функцією винагороди;
- модуль тестування та порівняльного аналізу методів керування;
- інтерактивний веб-застосунок Streamlit для демонстрації результатів.

Розроблена система дозволяє симулювати різні типи аварійних ситуацій включаючи одиночні відключення ліній та каскадні аварії типу N-1-1, порівнювати ефективність РРО агента з класичними алгоритмами та візуалізувати стан мережі і карту ризиків вузлів в реальному часі.

Висновки до розділу 1

У першому розділі сформульовано задачу керування топологією енергомережі та обґрунтовано її актуальність. Показано що традиційні підходи є недостатньо ефективними в умовах динамічних аварійних ситуацій. Задачу формалізовано як марківський процес прийняття рішень з простором станів розміром 114 параметрів та дискретним простором дій з 101 варіантом. Визначено математичну модель цільової функції оптимізації та обмеження на збереження зв'язності мережі. Обрано тестову мережу IEEE 14-bus як стандартний еталон для дослідження алгоритмів керування енергосистемами.

2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ

Перед розробкою власної системи керування топологією енергомережі було проведено аналіз існуючих підходів та інструментів. Аналіз охоплює три напрями: алгоритми керування енергомережами, інструменти симуляції та порівняльний аналіз підходів.

2.1 Огляд алгоритмів керування енергомережами

Існуючі підходи до керування топологією енергомереж можна розділити на три основні категорії: класичні алгоритми, методи математичної оптимізації та методи машинного навчання.

Класичні алгоритми базуються на заздалегідь визначених правилах. Найпростішим представником є жадібний алгоритм (greedy), який на кожному кроці відключає найбільш перевантажену лінію. Перевагою такого підходу є простота реалізації та швидкість прийняття рішень. Проте жадібний алгоритм має суттєвий недолік — він не враховує довгострокові наслідки своїх дій. Відключення перевантаженої лінії перерозподіляє навантаження на інші елементи мережі, що може спричинити каскадний збій. Як показали результати експериментів у даній роботі, жадібний алгоритм у критичних сценаріях виживає в середньому лише 2 кроки.

Методи математичної оптимізації, зокрема змішане цілочисельне лінійне програмування (MILP) [15], дозволяють знаходити оптимальне рішення з урахуванням усіх обмежень задачі. Такі методи широко використовуються в промислових системах керування енергомережами. Однак їх практичне застосування обмежене обчислювальною складністю — час розрахунку оптимального рішення для великих мереж може становити хвилини, тоді як реальні системи потребують прийняття рішень за секунди.

Методи машинного навчання, зокрема навчання з підкріпленням, є відносно новим підходом до керування енергомережами [6]. Починаючи з 2019 року компанія RTE France щорічно проводить змагання L2RPN (Learning to Run a Power Network) [1], метою яких є розробка агентів на основі навчання з підкріпленням для керування реалістичними моделями енергомереж. Результати змагань демонструють що агенти на основі PPO та інших алгоритмів навчання з підкріпленням здатні перевершувати класичні підходи в складних аварійних сценаріях [14].

2.1.1 Порівняння алгоритмів навчання з підкріпленням

Серед алгоритмів навчання з підкріпленням що застосовуються до задач керування енергомережами виділяють кілька основних підходів які відрізняються механізмом навчання та областю застосування.

Алгоритм DQN апроксимує функцію цінності дій за допомогою нейронної мережі [7]. Перевагою підходу є здатність працювати з великими просторами спостережень. Проте DQN має обмеження для задач керування енергомережею — алгоритм схильний до нестабільності навчання при великих просторах дій що є характерним для задач топологічного керування.

Алгоритм A3C використовує асинхронне навчання [11] кількох паралельних агентів що прискорює збіжність порівняно з однопотоковими методами. Однак складність налаштування гіперпараметрів та чутливість до вибору швидкості навчання обмежують його практичне застосування.

PPO усуває ключові недоліки попередніх підходів завдяки механізму кліпування функції втрат що обмежує величину оновлення політики на кожному кроці навчання [2]. Це забезпечує стабільне навчання без різких змін поведінки агента. Поєднання стабільності навчання зі здатністю працювати з дискретними просторами дій робить PPO найбільш придатним для задачі керування топологією енергомережі [18].

2.2 Огляд інструментів симуляції енергомереж

Для розробки та тестування алгоритмів керування енергомережами існує кілька спеціалізованих програмних інструментів.

Grid2Op — це відкрита Python бібліотека розроблена компанією RTE France [3] спеціально для задач навчання з підкріпленням в контексті керування енергомережами. Бібліотека надає стандартизований інтерфейс для взаємодії агента з симулятором, підтримує різні моделі мереж включаючи IEEE 14-bus, та забезпечує реалістичну симуляцію динаміки поточкорозподілу. Важливою перевагою Grid2Op є сумісність з бібліотеками машинного навчання через gym-сумісний інтерфейс.

Саме Grid2Op обрано як основний інструмент симуляції у даній роботі. Додатковою важливою характеристикою Grid2Op є реалістична модель захисних механізмів мережі. Симулятор автоматично відключає лінію якщо вона залишається перевантаженою більше двох кроків підряд — це відтворює поведінку релейного захисту реальних енергосистем. Така модель змушує агента навчитись запобігати перевантаженням завчасно а не реагувати на них після факту.

PandaPower — це Python бібліотека для аналізу та оптимізації електричних мереж. Вона надає широкий набір інструментів для розрахунку поточкорозподілу, аналізу стійкості та оптимізації режимів роботи мережі. PandaPower використовується як бекенд для розрахунків у Grid2Op, проте є повільнішим порівняно з LightSim2Grid.

LightSim2Grid — це швидкий математичний розв'язувач рівнянь поточкорозподілу розроблений як альтернативний бекенд для Grid2Op [9]. На відміну від PandaPower, LightSim2Grid реалізований на C++ що забезпечує значно вищу швидкість обчислень. У даній роботі використовується саме LightSim2Grid для прискорення процесу навчання агента. Практичне значення швидкості бекенду проявляється при навчанні агента протягом сотень тисяч кроків. У даній роботі навчання моделі протягом 300 000 кроків тривало приблизно 30 хвилин.

Скорочення часу одного циклу симуляції дозволяє проводити більше експериментів з різними конфігураціями гіперпараметрів за той самий проміжок часу що суттєво прискорює процес розробки.

PyPSA (Python for Power System Analysis) — це відкрита бібліотека для моделювання та оптимізації енергосистем. На відміну від Grid2Op, PyPSA орієнтована на довгострокове планування та оптимізацію енергосистем, а не на задачі керування в реальному часі. Тому PyPSA не розглядалась як інструмент для даної роботи.

Розглянуті інструменти симуляції відрізняються за призначенням, сумісністю з алгоритмами навчання з підкріпленням та швидкістю обчислень. У таблиці 2.1 наведено порівняльну характеристику розглянутих інструментів за цими критеріями.

Таблиця 2.1 — Порівняння інструментів симуляції енергомереж

Інструмент	Призначення	RL-сумісність	Швидкість симуляції
1	2	3	4
Grid2Op	Керування мережею в реальному часі	Так	Висока
PandaPower	Аналіз та оптимізація мереж	Обмежена	Середня
LightSim2Grid	Розрахунок поточкорозподілу	Так	Висока
PyPSA	Довгострокове планування	Ні	Середня

Як видно з таблиці 2.1, Grid2Op у поєднанні з LightSim2Grid є єдиною комбінацією що одночасно забезпечує повну сумісність з алгоритмами навчання з підкріпленням та високу швидкість симуляції. Це робить даний стек інструментів оптимальним вибором для реалізації системи керування топологією енергомережі.

2.3 Порівняльний аналіз підходів

На основі проведеного аналізу було визначено переваги та недоліки кожного підходу. У таблиці 2.2 наведено порівняння розглянутих методів керування енергомережами.

Таблиця 2.2 – Порівняння методів керування енергомережами

Метод	Швидкість	Якість рішення	Адаптивність	Складність реалізації
1	2	3	4	5
Жадібний алгоритм	Висока	Низька	Низька	Низька
MILP оптимізація	Низька	Висока	Низька	Висока
Random agent	Висока	Дуже низька	Відсутня	Дуже низька
PPO (навчання з підкріпленням)	Висока	Висока	Висока	Середня

Окрім характеристик наведених у таблиці 2.2, важливим критерієм є здатність до навчання на основі досвіду. Класичні алгоритми та MILP оптимізація є детермінованими — їхня поведінка повністю визначається заздалегідь запрограмованими правилами і не змінюється з часом незалежно від кількості оброблених сценаріїв. PPO агент навпаки покращує стратегію в процесі навчання і здатний виявляти нетривіальні рішення відсутні в класичних підходах.

Це підтверджується результатами порівняльного тестування проведеного в даній роботі. У важких аварійних сценаріях PPO агент стабільно перевершує класичні методи. Принципова перевага підходу полягає в тому що агент самостійно виробляє стратегію керування яка враховує довгострокові наслідки кожної дії — властивість якої принципово позбавлені жадібні алгоритми.

Як видно з таблиці 2.2, метод на основі PPO поєднує переваги швидкості та якості рішень, а його додатковою перевагою є адаптивність — навчений агент здатний виробляти нестандартні стратегії керування яких немає в класичних

алгоритмах. Саме тому PPO обрано основним методом для реалізації системи керування.

За результатами аналізу для реалізації системи керування обрано такі інструменти: Grid2Op як середовище симуляції, LightSim2Grid як швидкий розв'язувач поточкорозподілу, алгоритм PPO з бібліотеки Stable-Baselines3 як метод навчання агента, та Streamlit як інструмент розробки веб-застосунку.

Висновки до розділу 2

У другому розділі проведено аналіз існуючих підходів до керування топологією енергомереж за трьома категоріями: класичні алгоритми, методи математичної оптимізації та методи навчання з підкріпленням. Встановлено що класичні підходи ефективні лише в простих сценаріях і не здатні адаптуватись до нових умов, а методи оптимізації забезпечують якісні рішення але є непридатними для роботи в реальному часі через обчислювальну складність. Серед алгоритмів навчання з підкріпленням обґрунтовано перевагу PPO над DQN та A3C з точки зору стабільності навчання та придатності для дискретних просторів дій. Проведено огляд спеціалізованих інструментів симуляції енергомереж і обґрунтовано вибір Grid2Op з бекендом LightSim2Grid як основного середовища розробки. За результатами порівняльного аналізу сформовано стек інструментів для реалізації системи керування топологією енергомережі в реальному часі.

3 ЗАСОБИ РОЗРОБКИ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

У цьому розділі описано програмні засоби та бібліотеки що використовувались для реалізації системи керування топологією енергомережі. Вибір інструментів обумовлений їх широким застосуванням у науковому співтоваристві, відкритим вихідним кодом, активною підтримкою розробниками та взаємною сумісністю. Розглядаються середовище симуляції Grid2Op, алгоритм PPO та бібліотека Stable-Baselines3, мова програмування Python і веб-фреймворк Streamlit..

3.1 Середовище симуляції Grid2Op

Grid2Op — це відкрита Python бібліотека розроблена компанією RTE France для симуляції керування електричними мережами в контексті навчання з підкріпленням. Бібліотека була створена як основна платформа для міжнародних змагань L2RPN (Learning to Run a Power Network) що проводяться щорічно починаючи з 2019 року.

Основними компонентами Grid2Op є середовище симуляції що реалізує гнучкий інтерфейс, модуль розрахунку поточкорозподілу, система винагород та набори даних з часовими рядами навантаження. Бібліотека підтримує різні тестові мережі включаючи IEEE 14-bus яка використовується у даній роботі.

Grid2Op моделює роботу енергомережі покроково де кожен крок відповідає 5 хвилинам реального часу. На кожному кроці агент отримує спостереження про поточний стан мережі, обирає дію та отримує винагороду. Якщо будь-яка лінія залишається перевантаженою більше двох кроків підряд — симулятор автоматично відключає її імітуючи спрацювання захисного реле.

Для прискорення розрахунків у даній роботі використовується бекенд LightSim2Grid — реалізація розв'язувача рівнянь поточкорозподілу на мові C++.

Порівняно зі стандартним бекендом PandaPower, LightSim2Grid забезпечує вищу швидкість симуляції що суттєво скорочує час навчання агента.

3.1.1 Структура середовища Grid2Op

Середовище Grid2Op організоване за принципом взаємодії агента і середовища що відповідає стандартному інтерфейсу OpenAI Gym [13]. На кожному кроці симуляції відбувається така послідовність подій: середовище формує вектор спостережень що описує поточний стан мережі, агент отримує спостереження і обирає дію, середовище виконує дію і розраховує новий стан мережі, функція винагороди оцінює результат і повертає числовий сигнал агенту.

Часовий крок симуляції відповідає п'яти хвилинам реального часу енергосистеми. Це означає що навчання протягом 300 000 кроків моделює більше одного року роботи мережі в різних умовах навантаження. Набір з 20 сценаріїв навантаження що використовується у даній роботі охоплює різні режими споживання — від нічних мінімумів до денних піків — що забезпечує різноманітність навчальних ситуацій для агента.

3.2 Алгоритм PPO та бібліотека Stable-Baselines3

Proximal Policy Optimization (PPO) — це алгоритм навчання з підкріпленням розроблений компанією OpenAI у 2017 році [2]. PPO належить до класу алгоритмів Actor-Critic що використовують дві нейромережі: Actor для вибору дій та Critic для оцінки стану.

Ключовою особливістю PPO є механізм кліпування градієнтів що обмежує величину оновлення параметрів політики на кожному кроці навчання. Це запобігає різким змінам поведінки агента і забезпечує стабільність навчання. Математично кліпування реалізується через обмеження відношення нових та старих імовірностей дій у межах діапазону $[1-\epsilon, 1+\epsilon]$ де ϵ — параметр `clip_range`.

PPO обраний для даної роботи з кількох причин. По-перше, алгоритм добре працює з дискретним простором дій що відповідає задачі вибору однієї з 101 можливих конфігурацій ліній. По-друге, PPO є стандартним алгоритмом у змаганнях L2RPN що підтверджує його ефективність для задач керування енергомережами. По-третє, реалізація PPO у бібліотеці Stable-Baselines3 є добре задокументованою та перевіреною.

Stable-Baselines3 — це Python бібліотека що надає готові реалізації популярних алгоритмів навчання з підкріпленням включаючи PPO, A2C, SAC та інші [4]. Бібліотека побудована на основі PyTorch [16] та забезпечує зручний інтерфейс для навчання та тестування агентів. Для інтеграції з Grid2Op використовується модуль gym-сумісної обгортки що перетворює середовище Grid2Op у формат придатний для Stable-Baselines3.

3.2.1 Архітектура нейромережі PPO

У даній роботі використовується політика MlpPolicy що реалізує багатошаровий перцептрон як функціональний апроксиматор. Нейромережа складається з двох прихованих шарів по 64 нейрони з функцією активації ReLU. Така архітектура є стандартною для задач з невеликими векторними просторами спостережень і добре підходить для вектора з 114 параметрів що використовується у даній роботі.

PPO реалізує архітектуру Actor-Critic де дві окремі нейромережі поділяють спільні шари для обробки спостережень [12]. Actor формує розподіл імовірностей над простором дій і обирає дію для виконання. Critic оцінює поточний стан і передбачає очікувану сумарну винагороду що використовується для обчислення переваги — міри наскільки обрана дія є кращою або гіршою за середню. Спільне навчання обох мереж дозволяє агенту одночасно покращувати стратегію вибору дій і точність оцінки станів.

3.3 Мова програмування Python та веб-фреймворк Streamlit

Python — це інтерпретована мова програмування загального призначення що є стандартом де-факто для розробки систем машинного навчання та наукових обчислень. У даній роботі використовується Python версії 3.13. Основними перевагами Python для даної задачі є широка екосистема бібліотек для машинного навчання, зручний синтаксис та активна спільнота розробників.

Для розробки веб-застосунку використовується фреймворк Streamlit — відкрита Python бібліотека що дозволяє створювати інтерактивні веб-застосунки безпосередньо з Python коду [10] без необхідності знань HTML, CSS або JavaScript. Streamlit автоматично перетворює Python скрипти у веб-інтерфейс з підтримкою інтерактивних елементів керування, графіків та таблиць.

Застосунок Streamlit організований як однофайловий скрипт `app.py` що містить весь код інтерфейсу. При запуску Streamlit автоматично відстежує зміни у файлі і перезавантажує застосунок без необхідності ручного перезапуску. Це суттєво прискорює процес розробки та налагодження інтерфейсу.

Для збереження результатів між сесіями використовується механізм кешування Streamlit — декоратор `st.cache_data` дозволяє зберігати результати тривалих обчислень і завантажувати їх миттєво при повторному запиті. У даній роботі цей механізм застосовується для збереження результатів порівняльного тестування 20 сценаріїв що займає близько трьох хвилин при першому розрахунку.

Для візуалізації даних використовується бібліотека Matplotlib що надає широкі можливості для побудови статичних та інтерактивних графіків. Обробка числових даних здійснюється за допомогою бібліотеки NumPy, а табличні дані представляються з використанням Pandas.

Для логування процесу навчання використовується бібліотека TensorBoard що надає веб-інтерфейс для відстеження динаміки метрик навчання в реальному часі. Основними метриками що відстежувались у даній роботі є середня винагорода за епізод та середня тривалість епізоду. Аналіз цих метрик дозволив виявити три

характерні фази навчання — дослідження середовища, активне навчання та збіжність — і прийняти рішення про завершення навчання.

У таблиці 3.1 наведено повний перелік використаних програмних засобів та їх версії.

Таблиця 3.1 – Перелік використаних програмних засобів

Засіб	Версія	Призначення
1	2	3
Python	3.13	Мова програмування
Grid2Op	1.9+	Симуляція енергомережі
LightSim2Grid	0.7+	Розрахунок потокорозподілу
Stable-Baselines3	2.0+	Реалізація алгоритму PPO
Streamlit	1.41+	Веб-інтерфейс
Matplotlib	3.7+	Візуалізація даних
NumPy	1.23+	Числові обчислення
Pandas	1.4+	Обробка табличних даних

Висновки до розділу 3

У третьому розділі описано програмні засоби що використовувались для реалізації системи. Середовище Grid2Op з бекендом LightSim2Grid забезпечує реалістичну і швидку симуляцію енергомережі IEEE 14-bus з підтримкою гнучкого інтерфейсу. Алгоритм PPO з бібліотеки Stable-Baselines3 реалізує архітектуру Actor-Critic з механізмом кліпування градієнтів що забезпечує стабільне навчання. Нейромережева архітектура MlpPolicy з двома прихованими шарами по 64 нейрони є оптимальною для векторного простору спостережень розміром 114 параметрів. Веб-застосунок реалізовано на основі Streamlit з використанням механізму кешування для прискорення роботи з результатами

тестування. Обраний стек інструментів забезпечує необхідну функціональність для реалізації та демонстрації системи керування топологією енергомережі.

4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

Програмна система керування топологією енергомережі реалізована у вигляді набору взаємопов'язаних модулів мовою Python. Архітектура системи побудована за принципом розділення відповідальності — кожен модуль виконує чітко визначену функцію і може використовуватись незалежно.

4.1 Архітектура системи

Система складається з чотирьох основних компонентів: модуля налаштування середовища, модуля функції винагороди, модуля навчання агента та модуля оцінювання і візуалізації результатів. На рисунку 4.1 представлена загальна архітектура системи.

Модуль налаштування середовища (`env_setup.py`) відповідає за створення та конфігурацію симулятора `Grid2Op`. Він надає дві основні функції: `make_grid_env()` для створення середовища `Grid2Op` з заданими параметрами та `make_gym_env()` для перетворення середовища у `gym`-сумісний формат необхідний для навчання з бібліотекою `Stable-Baselines3`.

Модуль функції винагороди (`reward.py`) реалізує власну функцію винагороди `CustomReward` що успадковується від базового класу `BaseReward` бібліотеки `Grid2Op`. Модуль навчання (`train.py`) містить логіку навчання PPO агента з підтримкою логування через `TensorBoard`. Модуль оцінювання (`evaluate.py`) реалізує порівняльне тестування чотирьох методів: без агента, випадкового агента, жадібного алгоритму та PPO агента.

Структура проєкту організована наступним чином:

- `src/` — вихідний код системи (`env_setup.py`, `reward.py`, `train.py`, `evaluate.py`);
- `models/` — збережені навчені моделі агента;
- `outputs/` — результати тестування та візуалізації;

- `app.py` — головний файл веб-застосунку Streamlit.

Взаємодія між модулями системи організована за принципом однонаправленого потоку даних. Модуль `env_setup` створює середовище і передає його модулю `train` для навчання. Навчена модель зберігається у директорію `models` і завантажується модулем `evaluate` для тестування та веб-застосунком `app.py` для демонстрації. Модуль `reward` використовується як компонент середовища і не взаємодіє з іншими модулями безпосередньо. Загальну архітектуру системи та взаємодію між компонентами представлено на рисунку 4.1.

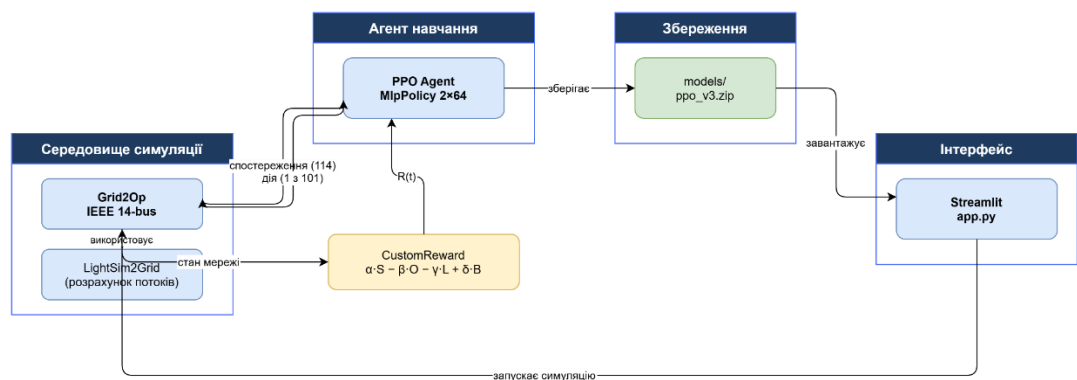


Рисунок 4.1 — компонентна діаграма

Такий підхід до організації коду забезпечує незалежність модулів — кожен з них може використовуватись окремо без необхідності запускати всю систему. Наприклад навчена модель може бути завантажена і протестована без повторного запуску процесу навчання що суттєво прискорює ітеративну розробку.

Функціональні можливості системи з точки зору взаємодії з користувачем відображено на діаграмі варіантів використання (рисунок 4.2).

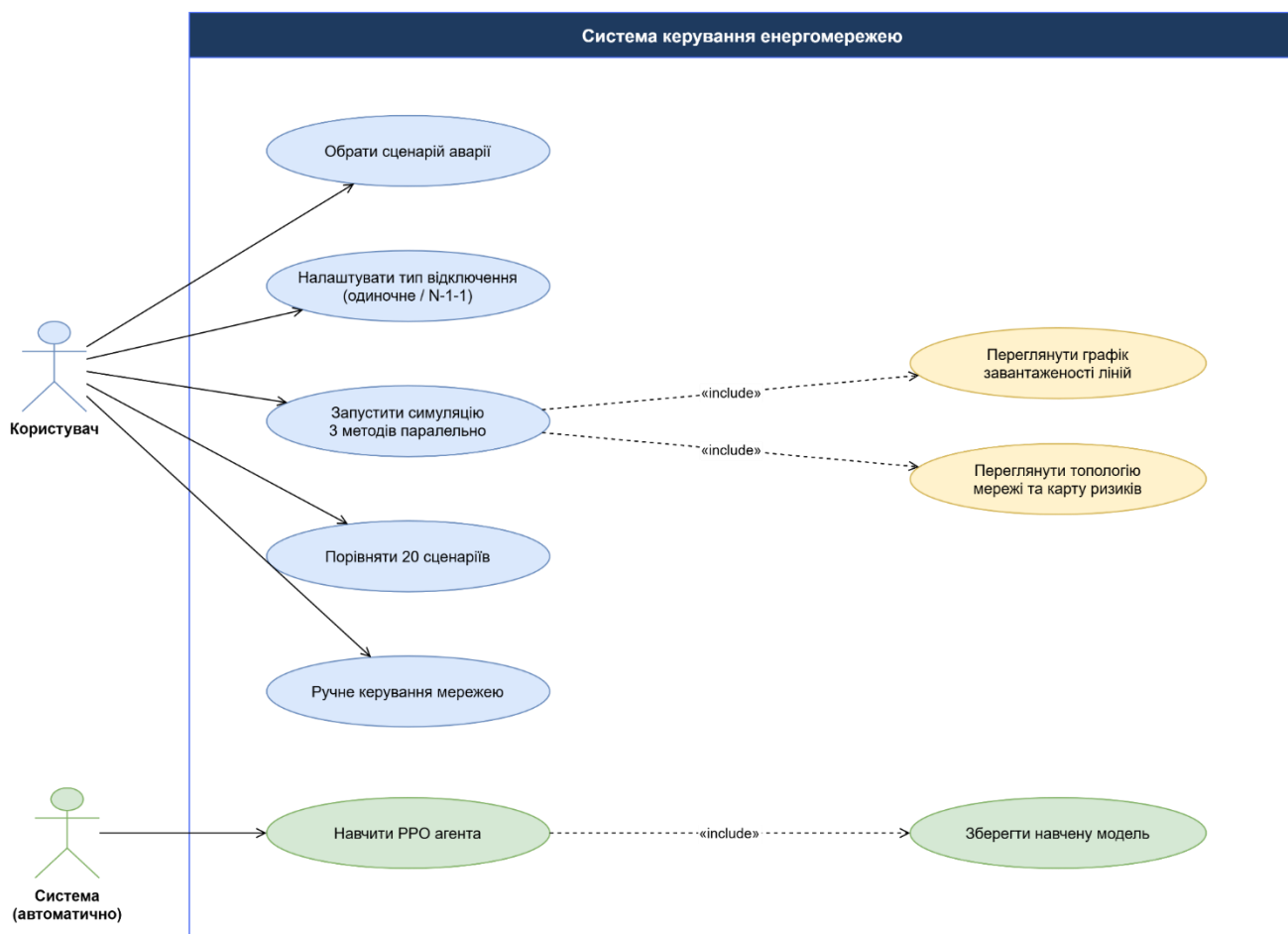


Рисунок 4.2 — Use Case діаграма

Діаграма відображає два типи акторів — користувача що взаємодіє з системою через веб-інтерфейс та автоматичні процеси навчання і збереження моделі що виконуються без участі користувача.

4.2 Математична модель задачі

Задача керування топологією енергомережі формалізована як марківський процес прийняття рішень (MDP) що описується кортежем (S, A, P, R, γ) [6], де S — простір станів, A — простір дій, P — функція переходів, R — функція винагороди, γ — коефіцієнт дисконтування.

Простір станів S представлений вектором спостережень розміром 114 параметрів. У таблиці 4.1 наведено складові вектора спостережень.

Таблиця 4.1 – Складові вектора спостережень агента

Параметр	Розмір	Опис
1	2	3
rho	20	Завантаженість кожної лінії (% від максимуму)
p_or	20	Активна потужність на початку лінії (МВт)
gen_p	6	Активна потужність генераторів (МВт)
load_p	11	Активна потужність навантажень (МВт)
topo_vect	57	Активна потужність навантажень (МВт)

Вектор спостережень формується автоматично середовищем Grid2Op на основі поточного стану мережі. Параметр rho є найважливішим для прийняття рішень — він безпосередньо відображає ризик перевантаження кожної лінії. Параметри gen_p та load_p надають агенту інформацію про баланс генерації та споживання що дозволяє прогнозувати зміни навантаження на лінії. Топологічний вектор topo_vect кодує поточну конфігурацію підключень вузлів до шин підстанцій що є необхідним для планування перемикачів.

Простір дій A є дискретним і містить 101 можливу дію:

- 1 дія — нічого не робити (do nothing);
- 40 дій — підключити або відключити кожну з 20 ліній передачі;
- 60 дій — перемикачів шин у підстанціях мережі.

Дія «нічого не робити» є важливою частиною простору дій — вона дозволяє агенту утримуватись від втручання коли мережа працює стабільно. Аналіз журналу дій навченого агента показав що у стабільних сценаріях агент обирає цю дію у переважній більшості кроків що свідчить про виважену стратегію мінімального втручання.

Функція переходів P описує динаміку мережі — як стан мережі змінюється після виконання дії. Ця функція реалізована внутрішньо симулятором Grid2Op через розв'язання рівнянь поточкорозподілу.

Коефіцієнт дисконтування $\gamma = 0.99$ означає що агент надає перевагу негайним винагородам але враховує і довгострокові наслідки своїх дій.

4.3 Реалізація функції винагороди

Функція винагороди є ключовим елементом системи що визначає поведінку агента в процесі навчання. У даній роботі розроблено власну функцію винагороди CustomReward що враховує чотири компоненти (формула 4.1):

$$R(t) = \alpha \cdot S(t) - \beta \cdot O(t) - \gamma \cdot L(t) + \delta \cdot B(t) \quad (4.1)$$

де $S(t)$ — компонент стабільності мережі,

$O(t)$ — компонент штрафу за перевантаження ліній,

$L(t)$ — компонент штрафу за блекаут,

$B(t)$ — компонент бонусу за збереження з'єднаності мережі,

$\alpha, \beta, \gamma, \delta$ — вагові коефіцієнти відповідних компонентів.

Компонент стабільності $S(t) = 1$ нараховується агенту за кожен крок роботи мережі без аварії. Це стимулює агента підтримувати мережу в робочому стані якомога довше.

Компонент штрафу за перевантаження $O(t)$ розраховується як сума перевищень допустимого порогу завантаженості по всіх активних лініях мережі. Поріг встановлений на рівні 90% від максимально допустимого навантаження що відповідає зоні підвищеного ризику.

Компонент штрафу за блекаут $L(t) = 1$ активується при повному відключенні мережі. Великий штрафний коефіцієнт $\gamma = 100$ гарантує що агент буде уникати блекауту за будь-яких умов.

Компонент бонусу за з'єднаність $B(t)$ розраховується як частка активних ліній від їх загальної кількості. Цей компонент стимулює агента підтримувати якомога більше активних з'єднань у мережі.

Вагові коефіцієнти $\alpha = 1.0$, $\beta = 2.0$, $\gamma = 100.0$, $\delta = 0.5$ підібрані емпірично в процесі налаштування системи. Значення коефіцієнтів відображають відносну важливість кожного компоненту: блекаут є найгіршим результатом ($\gamma = 100$), перевантаження важливіше за базову стабільність ($\beta = 2 > \alpha = 1$), а збереження з'єднаності є бажаним але менш критичним ($\delta = 0.5$).

Підбір вагових коефіцієнтів здійснювався методом послідовного налаштування. Спочатку було визначено пріоритет компонентів — блекаут є критичною подією тому його штраф встановлено на порядок вище за інші компоненти. Далі підбирався баланс між штрафом за перевантаження і бонусом за стабільність. Остаточні значення коефіцієнтів $\alpha = 1.0$, $\beta = 2.0$, $\gamma = 100.0$, $\delta = 0.5$ були зафіксовані після серії експериментів де порівнювались результати навчання з різними комбінаціями значень.

4.4 Навчання агента

Навчання PPO агента здійснювалось у три етапи що відповідають трьом версіям моделі. У таблиці 4.2 наведено характеристики кожної версії.

Таблиця 4.2 – Характеристики версій навченої моделі

Версія	Reward Function	Кроки навчання	Сер. кроків виживання	Критичний сценарій
1	2	3	4	5
v1	L2RPNReward	100 000	192	~13x
v2	L2RPNReward	200 000	1162	~518x
v3	CustomReward	300 000	2052	~1661x

Перша версія моделі (v1) навчалась із стандартною функцією винагороди L2RPNReward протягом 100 000 кроків. Результати показали що агент навчився

базовій стратегії виживання але демонстрував нестабільну поведінку в складних сценаріях.

Друга версія (v2) отримана шляхом продовження навчання до 200 000 кроків із тією самою функцією винагороди. Збільшення кількості кроків навчання дозволило агенту виробити більш стабільну стратегію — середня тривалість виживання зросла з 192 до 1162 кроків.

Третя версія (v3) навчалась із власною функцією винагороди CustomReward протягом 300 000 кроків. Використання власної функції винагороди дозволило досягти найкращих результатів — середня тривалість виживання зросла до 2052 кроків що на 77% перевищує результат v2.

Для навчання використовувались гіперпараметри наведені у таблиці 4.3.

Таблиця 4.3 – Гіперпараметри алгоритму PPO

Параметр	Значення	Призначення
1	2	3
learning_rate	3×10^{-4}	Швидкість навчання
n_steps	2048	Кількість кроків до оновлення політики
batch_size	64	Розмір міні-батчу
n_epochs	10	Кількість епох на одне оновлення
ent_coef	0.01	Коефіцієнт ентропії для заохочення дослідження
clip_range	0.2	Діапазон кліпування градієнтів
gamma	0.99	Коефіцієнт дисконтування майбутніх винагород

Вибір гіперпараметрів базувався на рекомендаціях документації Stable-Baselines3 та результатах попередніх досліджень із застосування PPO до задач керування енергомережами. Значення $\text{learning_rate} = 3 \times 10^{-4}$ є стандартним для

задач з безперервними спостереженнями і дискретними діями. Розмір буфера `n_steps = 2048` визначає кількість кроків що збираються перед кожним оновленням політики — більший буфер забезпечує стабільніші оцінки переваги але вимагає більше пам'яті. Коефіцієнт ентропії `ent_coef = 0.01` стимулює дослідження простору дій на початкових етапах навчання запобігаючи передчасній збіжності до субоптимальної стратегії.

Процес навчання логувався за допомогою TensorBoard що дозволило відстежувати динаміку основних метрик: середньої винагороди за епізод (`ep_rew_mean`) та середньої тривалості епізоду (`ep_len_mean`). Навчання моделі v3 тривало приблизно 30 хвилин на центральному процесорі без використання графічного прискорювача.

4.5 Тестування системи

Тестування розробленої системи проводилось на двох рівнях: модульне тестування окремих компонентів та системне тестування на повному наборі сценаріїв.

Модульне тестування функції винагороди `CustomReward` реалізовано у файлі `test_reward.py` з використанням бібліотеки `pytest`. Розроблено вісім тест-кейсів що перевіряють коректність кожного компоненту формули винагороди. У таблиці 4.4 наведено перелік реалізованих тестів та їх призначення.

Таблиця 4.4 — Перелік unit-тестів функції `CustomReward`

Назва тесту	Що перевіряє
1	2
<code>test_no_overload_no_blackout</code>	Позитивна винагорода при нормальній роботі
<code>test_blackout_gives_large_penalty</code>	Штраф -100 при блекауті

Продовження таблиці 4.4

1	2
test_overload_reduces_reward	Перевантаження зменшує винагороду
test_overload_threshold_boundary	Завантаженість 90% не дає штрафу
test_overload_above_threshold	Завантаженість понад 90% штрафується
test_connectivity_bonus	Більше активних ліній — вища винагорода
test_blackout_overrides_normal_state	Блекаут дає -100 незалежно від стану ліній
test_reward_components_sum	Коректність підрахунку суми компонентів

Усі вісім тестів пройшли успішно. Результати виконання тестів підтверджують що функція CustomReward коректно реалізує задану математичну модель винагороди.

Системне тестування проводилось шляхом автоматичного запуску симуляції на всіх 20 сценаріях навантаження з трьома типами аварій для кожного з чотирьох методів керування. Результати зберігались у файл формату JSON для подальшого аналізу. Порівняльний аналіз підтвердив стабільну перевагу PPO агента над класичними методами в аварійних сценаріях різної складності.

Висновки до розділу 4

У четвертому розділі описано програмну реалізацію системи керування топологією енергомережі. Представлено архітектуру системи з чотирьох взаємопов'язаних модулів та діаграми що відображають компонентну структуру і варіанти використання. Задачу формалізовано як MDP з вектором спостережень з 114 параметрів та дискретним простором з 101 дії. Реалізовано власну функцію винагороди CustomReward з чотирма компонентами та емпірично підібраними ваговими коефіцієнтами. Навчання агента проводилось у три ітерації з поступовим

покращенням результатів — від 192 кроків виживання у першій версії до 2052 у третій. Проведено модульне тестування функції винагороди з вісьмома unit-тестами та системне тестування на 20 сценаріях навантаження.

5 РОБОТА КОРИСТУВАЧА З ПРОГРАМНОЮ СИСТЕМОЮ

Програмна система реалізована у вигляді веб-застосунку на основі фреймворку Streamlit та запускається локально командою `streamlit run app.py`. Після запуску застосунок автоматично відкривається у браузері за адресою <http://localhost:8501>. Інтерфейс застосунку організований у вигляді чотирьох вкладок: "Симуляція", "Порівняння сценаріїв", "Архітектура" та "Ручне керування".

5.1 Вимоги до середовища та встановлення

Для запуску програмної системи необхідне встановлене середовище Python версії 3.10 або вище. Рекомендується використовувати віртуальне середовище для ізоляції залежностей проєкту від системних бібліотек. Встановлення всіх необхідних залежностей виконується однією командою з файлу `requirements.txt`.

Запуск веб-застосунку здійснюється командою `streamlit run app.py` з кореневої директорії проєкту. Після запуску застосунок автоматично відкривається у браузері за адресою <http://localhost:8501>. Зупинка застосунку виконується комбінацією клавіш `Ctrl+C` у терміналі.

Перед першим запуском рекомендується виконати попереднє обчислення результатів порівняння сценаріїв командою `python src/evaluate.py`. Це дозволить завантажувати результати тестування миттєво з кешу замість очікування трьох хвилин при першому відкритті вкладки порівняння сценаріїв.

5.2 Вкладка симуляції аварій

Вкладка "Симуляція" є основним робочим інструментом системи що дозволяє запускати симуляцію аварійних сценаріїв та порівнювати ефективність трьох методів керування.

Бічна панель керування містить такі елементи:

- випадальний список для вибору одного з 20 сценаріїв навантаження з автоматичною класифікацією на стабільні, важкі та критичні;
- перемикач типу аварії з трьома варіантами: без аварії, одиночне відключення лінії та каскадна аварія N-1-1;
- випадальні списки для вибору ліній що відключаються;
- повзунок для налаштування кількості кроків симуляції від 20 до 200;
- кнопка запуску симуляції.

Після натискання кнопки система одночасно запускає симуляцію трьох методів на однакових умовах: без агента, жадібного алгоритму та PPO агента. Результати відображаються у вигляді чотирьох метрик: кількість кроків виживання для кожного методу та відношення PPO до жадібного алгоритму (рисунок 5.1).

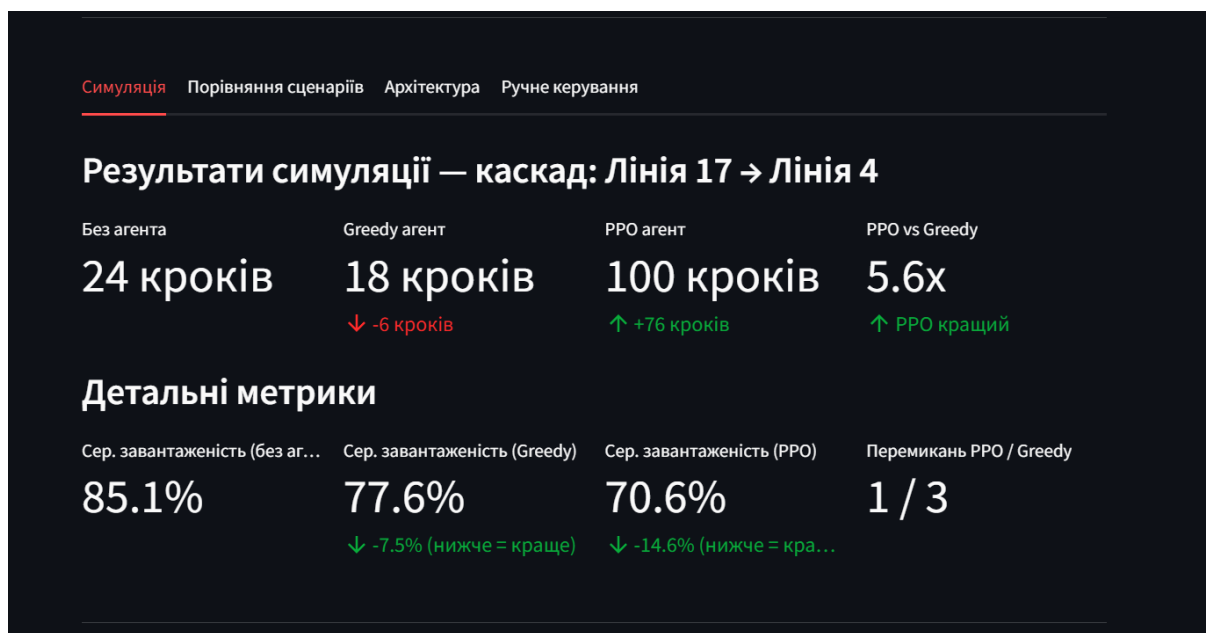


Рисунок 5.1 – Результати симуляції каскадної аварії N-1-1

Детальні метрики включають середню завантаженість ліній для кожного методу та кількість перемикачів виконаних агентами. Нижча середня завантаженість свідчить про кращу якість керування мережею.

Графік максимальної завантаженості ліній у часі дозволяє візуально оцінити динаміку роботи кожного методу (рисунок 5.2). На графіку відображаються три лінії: червона — без агента, жовта пунктирна — жадібний алгоритм, синя — PPO агент. Горизонтальна пунктирна лінія позначає межу перевантаження 100%.

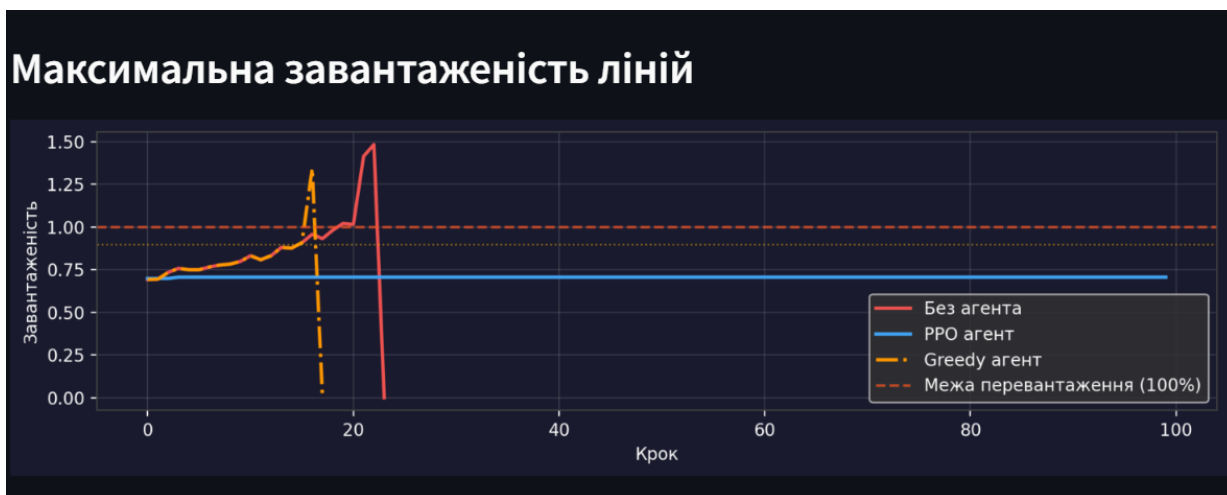


Рисунок 5.2 – Графік максимальної завантаженості ліній

Журнал дій PPO агента відображає всі перемикання ліній виконані агентом із зазначенням номеру кроку, назви лінії, типу дії та максимального навантаження на момент дії (рисунок 5.3).

Журнал дій PPO агента

Крок	Лінія	Дія	Макс. завантаженість
3	Лінія 1	✗ відключено	136.1%

Всього дій: 1 | Підключень: 0 | Відключень: 1

Рисунок 5.3 – Журнал дій агента

Журнал дій є важливим інструментом для розуміння стратегії агента. Аналіз журналу дозволяє виявити закономірності у поведінці агента — зокрема які лінії він відключає найчастіше і на яких кроках симуляції агент найбільш активний. У стабільних сценаріях журнал зазвичай містить одну-дві дії або залишається порожнім що свідчить про стратегію мінімального втручання. У критичних сценаріях агент може виконувати кілька послідовних перемикань для перерозподілу навантаження.

Топологія мережі у фінальному стані відображається для кожного методу окремо у вигляді графу (рисунок 5.4). Вузли позначаються жовтими зірочками для генераторів та синіми кружечками для навантажень. Лінії передачі кодуються кольором залежно від рівня завантаженості: зелений — норма до 70%, жовтий —

зона ризику 70-90%, червоний — небезпека понад 90%, пунктир — відключена лінія.

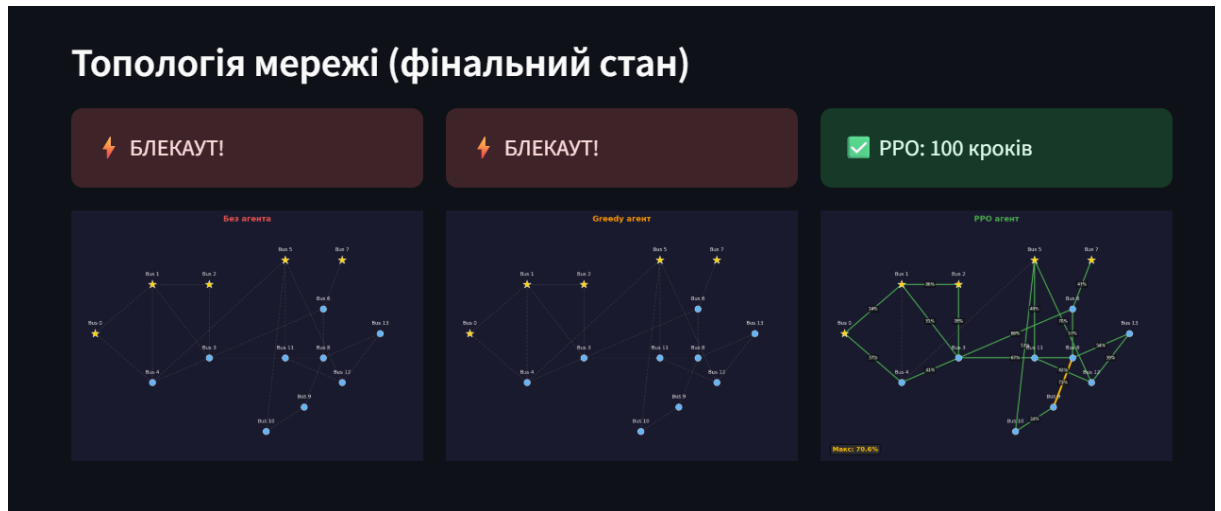


Рисунок 5.4 – Топологія мережі у фінальному стані

Колірне кодування ліній передачі є ключовим елементом візуалізації топології. Зелений колір відповідає безпечному рівню завантаженості до 70% і свідчить про нормальну роботу лінії. Жовтий колір сигналізує про підвищений рівень ризику в діапазоні 70-90% — такі лінії потребують уваги але не є критичними. Червоний колір позначає лінії з завантаженістю понад 90% що знаходяться в зоні небезпеки і можуть бути автоматично відключені симулятором. Пунктирна лінія позначає відключені елементи мережі. Порівняння топологій трьох методів на одному екрані дозволяє наочно оцінити різницю у стратегіях керування — зокрема побачити які саме лінії відключив жадібний алгоритм що призвело до блекауту і як PPO агент уникнув цього результату.

Карта ризиків вузлів відображає вразливість кожного вузла мережі після аварії у вигляді теплової карти (рисунок 5.5). Розмір та колір кожного вузла пропорційні середньому навантаженню на прилеглі лінії. Під картою відображаються три найризикованіші вузли з відсотком ризику.

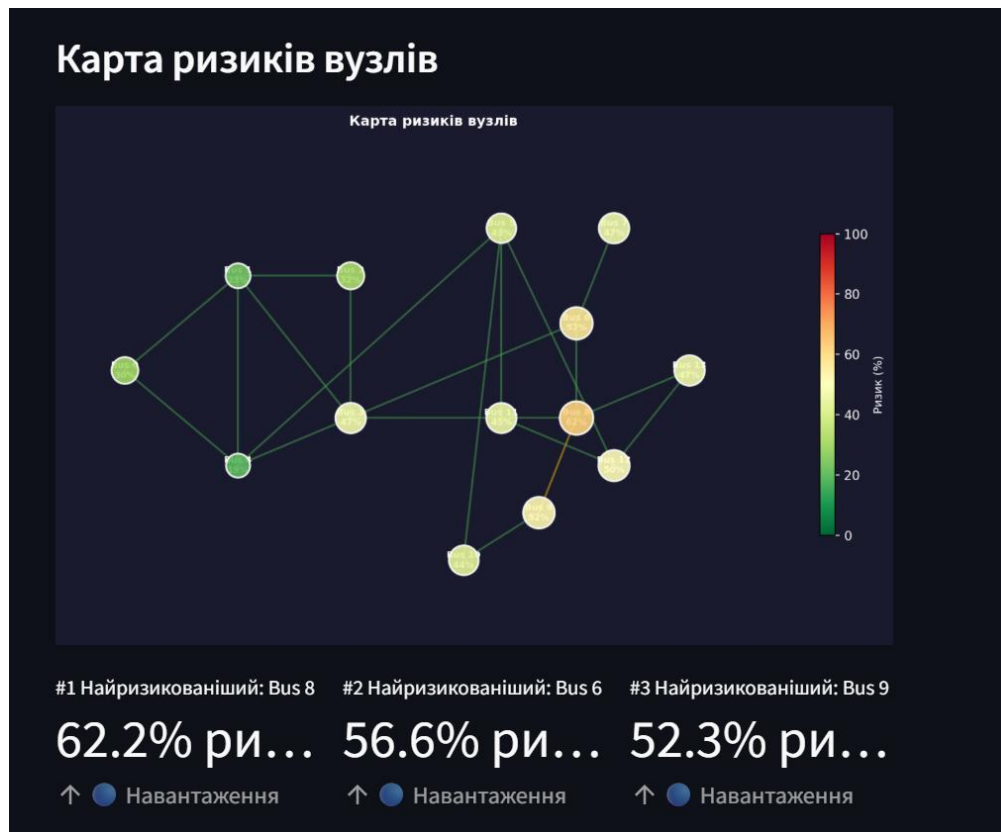


Рисунок 5.5 – Карта ризиків вузлів мережі

Карта ризиків доповнює топологічне представлення кількісною оцінкою вразливості кожного вузла. На відміну від графу топології який показує поточний стан ліній карта ризиків відображає агреговану інформацію про середнє навантаження на всі лінії що підключені до кожного вузла. Три найризикованіші вузли виводяться під картою з числовим значенням відсотку ризику що дозволяє оператору зосередити увагу на найвразливіших елементах мережі. Ця інформація може використовуватись для планування профілактичних заходів або додаткового резервування критичних вузлів.

5.3 Порівняння сценаріїв

Вкладка "Порівняння сценаріїв" забезпечує автоматичне тестування всіх 20 сценаріїв навантаження та відображення зведених результатів.

Вкладка містить дві кнопки: завантаження попередньо збережених результатів з кешу та перерахунок результатів заново. Завантаження з кешу

відбувається миттєво оскільки результати зберігаються у файлі формату JSON після першого розрахунку. Перерахунок займає приблизно 3 хвилини.

Результати відображаються у вигляді таблиці де кожен рядок відповідає одному сценарію (рисунок 5.6). Таблиця містить колонки: номер сценарію, тип (стабільний, важкий, критичний), кількість кроків виживання для кожного методу та відношення PPO до базових методів.

Сценарій	Тип	Без агента	Greedy	PPO агент	PPO vs без агента	PPO vs Greedy
0	стабільний	300	300	300	1	1
1	стабільний	300	2	300	1	150
2	стабільний	300	300	300	1	1
3	критичний ⚠	3	1	3	1	3
4	стабільний	300	300	300	1	1
5	стабільний	300	300	300	1	1
6	стабільний	300	2	300	1	150
7	стабільний	300	300	300	1	1
8	стабільний	300	300	300	1	1
9	стабільний	300	300	300	1	1

Рисунок 5.6 – Таблиця порівняння всіх 20 сценаріїв

Стовпчиковий графік відображає результати всіх сценаріїв в графічному вигляді (рисунок 5.7). Три стовпці різних кольорів відповідають трьом методам: червоний — без агента, помаранчевий — жадібний алгоритм, синій — PPO агент.



Рисунок 5.7 – Графік порівняння методів по всіх сценаріях

5.4 Архітектура системи

Вкладка "Архітектура" містить технічну документацію системи у зручному для перегляду форматі. Вкладка організована у два стовпці.

Лівий стовпець вкладки містить математичну документацію функції винагороди. Формула CustomReward представлена у символічному вигляді з розшифруванням кожного компоненту та його вагового коефіцієнту. Правий стовпець містить порівняльну таблицю трьох версій навченої моделі що наочно демонструє прогрес навчання від версії v1 до v3. Вигляд верхньої частини вкладки наведено на рисунку 5.8.

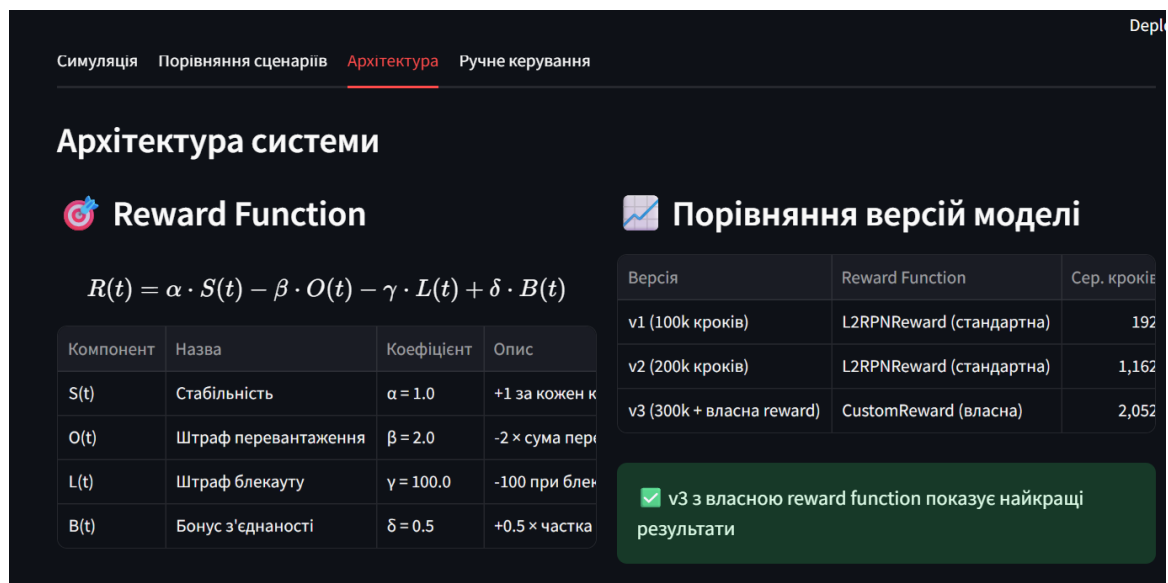


Рисунок 5.8 — скрін 1 з формулою і порівнянням версій

Нижня частина вкладки містить таблицю гіперпараметрів алгоритму PPO та технічні характеристики нейромережі MlpPolicy. Характеристики нейромережі включають тип архітектури Actor-Critic, кількість прихованих шарів і нейронів, функцію активації, розміри входу та виходу для кожної мережі, а також загальну кількість параметрів моделі — близько 30 000. Вигляд нижньої частини вкладки наведено на рисунку 5.9



Рисунок 5.9 — скрін 2 з гіперпараметрами і нейромережею

Наявність технічної документації безпосередньо в інтерфейсі застосунку робить систему самодокументованою — користувач може ознайомитись з усіма технічними деталями без звернення до зовнішньої документації.

5.5 Ручне керування

Вкладка "Ручне керування" реалізує інтерактивний режим де користувач може самостійно керувати конфігурацією ліній мережі та порівнювати свій результат з PPO агентом (рисунок 5.10).

Панель налаштувань містить вибір сценарію, повзунок кількості кроків та список з прапорцями для кожної з 20 ліній мережі. Знімаючи прапорець користувач відключає відповідну лінію перед початком симуляції.

Після запуску система паралельно виконує симуляцію з вибраною конфігурацією ліній та симуляцію PPO агента. Результати відображаються у вигляді трьох метрик: результат користувача, результат PPO агента та переможець з різницею в кроках.

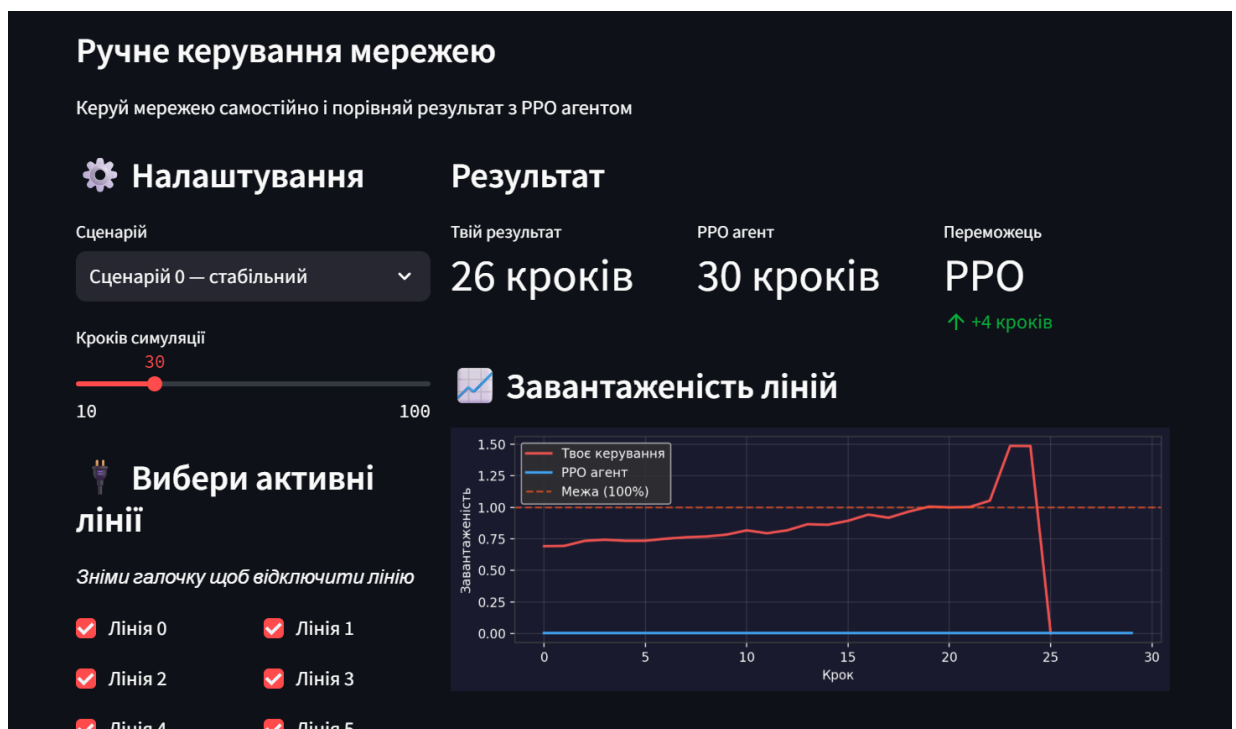


Рисунок 5.10 – Режим ручного керування мережею

Графік завантаженості ліній та топологія мережі дозволяють порівняти якість керування користувача та РРО агента. Як правило РРО агент демонструє нижчу середню завантаженість навіть при однаковій тривалості симуляції що свідчить про вищу якість керування.

5.6 Розгортання та подальший розвиток системи

Розроблена система призначена для локального запуску на персональному комп'ютері користувача. Для розгортання достатньо скопіювати директорію проєкту і встановити залежності з файлу requirements.txt. Навчена модель включена до складу проєкту у директорії models що дозволяє одразу використовувати систему без повторного навчання агента.

Для розгортання на віддаленому сервері або хмарній платформі застосунок Streamlit може бути опублікований через сервіс Streamlit Community Cloud що надає безкоштовний хостинг для відкритих проєктів. Це дозволяє забезпечити доступ до системи через веб-браузер без необхідності локального встановлення.

Подальший розвиток системи може включати кілька напрямків. По-перше, масштабування на мережі більшого розміру — наприклад IEEE 118-bus що містить 118 вузлів і 186 ліній передачі. По-друге, порівняння з додатковими алгоритмами навчання з підкріпленням такими як SAC або TD3 що можуть показати кращі результати на більш складних топологіях. По-третє, інтеграція з реальними даними навантаження для підвищення практичної цінності системи.

Висновки до розділу 5

У п'ятому розділі описано порядок роботи користувача з програмною системою. Визначено системні вимоги та процес встановлення і запуску застосунку. Описано функціональні можливості чотирьох вкладок веб-інтерфейсу: симуляція аварійних сценаріїв з порівнянням трьох методів керування,

автоматичне тестування всіх 20 сценаріїв навантаження, перегляд архітектури системи та режим ручного керування мережею. Розглянуто варіанти розгортання системи та визначено перспективи подальшого розвитку включаючи масштабування на мережі більшого розміру та порівняння з додатковими алгоритмами навчання з підкріпленням.

ВИСНОВКИ

У дипломній роботі розроблено програмну систему керування топологією енергомережі IEEE 14-bus на основі алгоритму глибокого навчання з підкріпленням PPO. Отримані результати підтверджують ефективність запропонованого підходу для задачі запобігання блекаутам в умовах аварійних відключень.

У процесі виконання роботи отримано такі результати:

Проведено аналіз існуючих підходів до керування топологією енергомереж. Встановлено що класичні алгоритми є ефективними лише в простих сценаріях і не здатні адаптуватись до нових умов, методи математичної оптимізації забезпечують якісні рішення але непридатні для роботи в реальному часі, а методи навчання з підкріпленням поєднують швидкість і якість прийняття рішень. Обґрунтовано вибір алгоритму PPO як найбільш ефективного для задач керування в реальному часі з дискретним простором дій.

Розроблено математичну модель задачі у вигляді марківського процесу прийняття рішень. Простір станів представлений вектором з 114 параметрів що описують поточний стан мережі, простір дій містить 101 дискретний варіант керування топологією. Визначено цільову функцію оптимізації та обмеження на збереження зв'язності мережі.

Реалізовано власну функцію винагороди CustomReward що враховує чотири компоненти: стабільність мережі, перевантаження ліній, блекаут та збереження з'єднаності. Вагові коефіцієнти підібрані емпірично і відображають відносну важливість кожного компоненту — штраф за блекаут з коефіцієнтом 100 є домінуючим що гарантує пріоритет уникнення повного відключення мережі.

Навчено три версії PPO агента з поступовим покращенням результатів. Перша версія з стандартною функцією винагороди за 100 000 кроків навчання досягла середньої тривалості виживання 192 кроки. Друга версія за 200 000 кроків — 1 162 кроки. Третя версія з власною функцією CustomReward за 300 000 кроків

досягла 2 052 кроки що на 77% перевищує результат другої версії. Весь процес навчання тривав близько 30 хвилин на центральному процесорі.

Проведено порівняльне тестування чотирьох методів керування на 20 сценаріях навантаження різної складності. PPO агент стабільно перевершує класичні підходи в аварійних сценаріях. У важких сценаріях перевага агента над жадібним алгоритмом сягає сотень разів що демонструє принципову відмінність між підходами на основі навчання та детермінованими алгоритмами.

Розроблено інтерактивний веб-застосунок Streamlit з чотирма вкладками що забезпечує симуляцію аварійних сценаріїв, порівняння методів керування, відображення технічної документації системи та режим ручного керування мережею. Застосунок запускається локально однією командою і не потребує додаткового налаштування.

Практична цінність розробленої системи полягає у демонстрації ефективності глибокого навчання з підкріпленням як підходу до керування топологією енергомережі. Система є proof of concept що може бути адаптований до реальних енергосистем більшого масштабу. Подальший розвиток може включати перенавчання агента на мережах типу IEEE 118-bus, порівняння з додатковими алгоритмами навчання з підкріпленням та інтеграцію з реальними даними навантаження.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Marot A., Donnot B., Romero C. et al. Learning to run a power network challenge for training topology-aware agents. Electric Power Systems Research. 2020. Vol. 189. URL: <https://doi.org/10.1016/j.epsr.2020.106635>
2. Schulman J., Wolski F., Dhariwal P. et al. Proximal Policy Optimization Algorithms. arXiv preprint. 2017. URL: <https://arxiv.org/abs/1707.06347>
3. Grid2Op documentation. RTE France, 2023. URL: <https://grid2op.readthedocs.io> (дата звернення: 11.05.2026).
4. Stable-Baselines3 documentation. 2023. URL: <https://stable-baselines3.readthedocs.io> (дата звернення: 11.05.2026).
5. Christie R. Power Systems Test Case Archive. University of Washington, 1999. URL: <https://labs.ece.uw.edu/pstca> (дата звернення: 11.05.2026).
6. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. Cambridge : MIT Press, 2018. 526 p.
7. Mnih V., Kavukcuoglu K., Silver D. et al. Human-level control through deep reinforcement learning. Nature. 2015. Vol. 518. P. 529–533.
8. Glavic M., Fonteneau R., Ernst D. Reinforcement Learning for Electric Power System Decision and Control: Past Considerations and Prospects. IFAC-PapersOnLine. 2017. Vol. 50, No 1. P. 6918–6927.
9. LightSim2Grid documentation. RTE France, 2023. URL: <https://lightsim2grid.readthedocs.io> (дата звернення: 11.05.2026).
10. Streamlit documentation. Streamlit Inc., 2023. URL: <https://docs.streamlit.io> (дата звернення: 11.05.2026).
11. Mnih V., Badia A. P., Mirza M. et al. Asynchronous Methods for Deep Reinforcement Learning. Proceedings of the 33rd International Conference on Machine Learning. 2016. Vol. 48. P. 1928–1937. URL: <https://arxiv.org/abs/1602.01783>
12. Konda V., Tsitsiklis J. Actor-Critic Algorithms. Advances in Neural Information Processing Systems. 2000. Vol. 13. P. 1008–1014

13. Brockman G., Cheung V., Pettersson L. et al. OpenAI Gym. arXiv preprint. 2016. URL: <https://arxiv.org/abs/1606.01540>
14. Marot A., Kelly A., Naglic M. et al. Perspectives on Future Power System Control Centers for Energy Transition. Journal of Modern Power Systems and Clean Energy. 2022. Vol. 10, No 2. P. 328–344. URL: <https://doi.org/10.35833/MPCE.2021.000673>
15. Capitanescu F., Martinez Ramos J. L., Panciatici P. et al. State-of-the-art, challenges, and future trends in security constrained optimal power flow. Electric Power Systems Research. 2011. Vol. 81, No 8. P. 1731–1741.
16. Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems. 2019. Vol. 32. URL: <https://arxiv.org/abs/1912.01703>
17. Grainger J. J., Stevenson W. D. Power Systems Analysis. New York : McGraw-Hill, 1994. 787 p.
18. Marot A., Donnot B., Dulac-Arnold G. et al. Learning to run a power network challenge: a retrospective analysis. arXiv preprint. 2021. URL: <https://arxiv.org/abs/2103.03104>

ДОДАТОК А Лістинг розробленої системи

```
# reward.py — власна функція винагороди
import numpy as np
from grid2op.Reward import BaseReward
```

```
class CustomReward(BaseReward):
```

```
    """
```

Власна функція винагороди для керування топологією енергомережі.

Математична модель:

$$R(t) = \alpha \cdot S(t) - \beta \cdot O(t) - \gamma \cdot L(t) + \delta \cdot B(t)$$

де:

$S(t)$ — винагорода за стабільність мережі

$O(t)$ — штраф за перевантаження ліній

$L(t)$ — штраф за блекаут

$B(t)$ — бонус за збереження з'єднаності

```
    """
```

```
def __init__(self):
```

```
    super().__init__()
```

```
    # Коефіцієнти функції винагороди
```

```
    self.alpha = 1.0 # вага стабільності
```

```
    self.beta = 2.0 # вага штрафу за перевантаження
```

```
    self.gamma = 100.0 # вага штрафу за блекаут
```

```
    self.delta = 0.5 # вага бонусу за з'єднаність
```

```
def initialize(self, env):
```

```

self.reward_min = -self.gamma
self.reward_max = self.alpha + self.delta

def __call__(self, action, env, has_error, is_done,
             is_illegal, is_ambiguous):

    if is_done:
        # Блекаут — максимальний штраф
        return -self.gamma

    obs = env.get_obs()

    # S(t) — базова винагорода за виживання
    s_t = self.alpha

    # O(t) — штраф за перевантаження
    # Рахуємо суму перевантажень по всіх активних лініях
    overloaded = obs.rho[obs.line_status] - 0.9
    overloaded = np.clip(overloaded, 0, None) # тільки > 90%
    o_t = self.beta * np.sum(overloaded)

    # B(t) — бонус за з'єднаність мережі
    # Більше активних ліній = краще
    connectivity = np.sum(obs.line_status) / len(obs.line_status)
    b_t = self.delta * connectivity

    reward = s_t - o_t + b_t

    return float(reward)

```

```

env_setup.py

import grid2op
from lightsim2grid import LightSimBackend
from grid2op.Parameters import Parameters
from grid2op.gym_compat import GymEnv, BoxGymObsSpace,
DiscreteActSpace
from src.reward import CustomReward

def make_grid_env(scenario_id=None):
    """Створює grid2op середовище з власною функцією винагороди"""
    param = Parameters()
    param.NO_OVERFLOW_DISCONNECTION = False
    param.NB_Timestep_OVERFLOW_ALLOWED = 2

    env = grid2op.make(
        "l2rpn_case14_sandbox",
        backend=LightSimBackend(),
        param=param,
        reward_class=CustomReward # власна reward function
    )

    if scenario_id is not None:
        env.reset(options={"time serie id": scenario_id})

    return env

def make_gym_env(env):
    """Обгортає grid2op env у gym-сумісний формат для РРО"""

```

```

gym_env = GymEnv(env)
gym_env.observation_space = BoxGymObsSpace(
    env.observation_space,
    attr_to_keep=["rho", "p_or", "gen_p", "load_p", "topo_vect"]
)
gym_env.action_space = DiscreteActSpace(
    env.action_space,
    attr_to_keep=["set_line_status"]
)
return gym_env

```

train.py

```

from src.env_setup import make_grid_env, make_gym_env
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import BaseCallback
import numpy as np

```

```

class TrainingCallback(BaseCallback):
    """Логую прогрес навчання"""
    def __init__(self, verbose=0):
        super().__init__(verbose)
        self.episode_rewards = []
        self.episode_lengths = []

    def _on_step(self):
        if self.locals.get("dones", [False])[0]:
            info = self.locals.get("infos", [{}])[0]
            ep_info = info.get("episode")

```



```

        if ep_info:
            self.episode_rewards.append(ep_info["r"])
            self.episode_lengths.append(ep_info["l"])
        return True

def train(timesteps=300_000, model_name="ppo_grid2op_v3"):
    print("=" * 50)
    print("Навчання PPO з власною Reward Function")
    print(f"Кроків: {timesteps:,}")
    print("=" * 50)

    env = make_grid_env()
    gym_env = make_gym_env(env)

    print("\nПростір спостережень (Observation Space):")
    print(f"Розмір: {gym_env.observation_space.shape}")
    print(f"Параметри: rho, p_or, gen_p, load_p, topo_vect")

    print("\nПростір дій (Action Space):")
    print(f"Розмір: {gym_env.action_space.n} дискретних дій")
    print(f"Тип: перемикання статусів ліній")

    print("\nФункція винагороди (Reward Function):")
    print(f" $R(t) = \alpha \cdot S(t) - \beta \cdot O(t) - \gamma \cdot L(t) + \delta \cdot B(t)$ ")
    print(f" $\alpha=1.0$  (стабільність),  $\beta=2.0$  (перевантаження)")
    print(f" $\gamma=100.0$  (блекаут),  $\delta=0.5$  (з'єднаність)")

    # Гіперпараметри PPO
    hyperparams = {

```

```

        "learning_rate": 3e-4,
        "n_steps": 2048,
        "batch_size": 64,
        "n_epochs": 10,
        "ent_coef": 0.01,
        "clip_range": 0.2,
        "gamma": 0.99,
    }

    print("\nГіперпараметри PPO:")
    for k, v in hyperparams.items():
        print(f" {k}: {v}")

    model = PPO(
        "MlpPolicy",
        gym_env,
        verbose=1,
        tensorboard_log="./tensorboard_logs/",
        **hyperparams
    )

    callback = TrainingCallback()

    print("\nПочинаємо навчання...")
    model.learn(
        total_timesteps=timesteps,
        callback=callback,
        tb_log_name="PPO_v3_custom_reward"
    )

```

```

model.save(model_name)
print(f"\nМодель збережена: {model_name}.zip")

if callback.episode_rewards:
    print(f"Середня винагорода (останні 10 епізодів): "
          f"{np.mean(callback.episode_rewards[-10:]):.2f}")
    print(f"Середня довжина епізоду: "
          f"{np.mean(callback.episode_lengths[-10:]):.0f} кроків")

return model

if __name__ == "__main__":
    train()

```

evaluate.py

```

import numpy as np
from src.env_setup import make_grid_env, make_gym_env
from stable_baselines3 import PPO

def run_episode_no_agent(env):
    obs = env.reset()
    done, steps, total_reward = False, 0, 0
    while not done:
        obs, reward, done, _ = env.step(env.action_space({}))
        total_reward += reward

```

```

        steps += 1
    return steps, total_reward

def run_episode_agent(gym_env, model):
    obs, _ = gym_env.reset()
    done, steps, total_reward = False, 0, 0
    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, terminated, truncated, _ = gym_env.step(action)
        total_reward += reward
        steps += 1
        done = terminated or truncated
    return steps, total_reward

def run_episode_random(gym_env):
    obs, _ = gym_env.reset()
    done, steps, total_reward = False, 0, 0
    while not done:
        action = gym_env.action_space.sample()
        obs, reward, terminated, truncated, _ = gym_env.step(action)
        total_reward += reward
        steps += 1
        done = terminated or truncated
    return steps, total_reward

def run_episode_greedy(env):
    """Жадібний агент — відключає найперевантаженішу лінію"""
    obs = env.reset()
    done, steps, total_reward = False, 0, 0

```

```

while not done:
    rho = obs.rho.copy()
    rho[~obs.line_status] = 0
    if np.max(rho) > 0.9:
        line_to_disconnect = np.argmax(rho)
        action = env.action_space(
            {"set_line_status": [(int(line_to_disconnect), -1)]}
        )
    else:
        action = env.action_space({})
    obs, reward, done, _ = env.step(action)
    total_reward += reward
    steps += 1
return steps, total_reward

```

```

def evaluate(model_name="ppo_grid2op_v3", n_episodes=5):
    env = make_grid_env()
    gym_env = make_gym_env(env)
    model = PPO.load(model_name, env=gym_env)

    print(f"Тестування {n_episodes} епізодів...\n")

    no_steps, no_rewards = [], []
    ag_steps, ag_rewards = [], []
    rnd_steps, rnd_rewards = [], []
    grd_steps, grd_rewards = [], []

    for i in range(n_episodes):
        s, r = run_episode_no_agent(env)

```

```

no_steps.append(s); no_rewards.append(r)

s, r = run_episode_agent(gym_env, model)
ag_steps.append(s); ag_rewards.append(r)

s, r = run_episode_random(gym_env)
rnd_steps.append(s); rnd_rewards.append(r)

env_g = make_grid_env()
s, r = run_episode_greedy(env_g)
grd_steps.append(s); grd_rewards.append(r)

print(f'Епізод {i+1}: без агента={no_steps[-1]:4d} | "
      f"random={rnd_steps[-1]:4d} | "
      f"greedy={grd_steps[-1]:4d} | "
      f"PPO={ag_steps[-1]:4d}")

print("\n=== ПІДСУМОК ===")
print(f'{"Метод":<15} {"Кроки (сер.):<15} {"Винагорода (сер.):"}')
      print(f'{"Без агента":<15} {np.mean(no_steps):<15.0f}
{np.mean(no_rewards):.0f}')
      print(f'{"Random agent":<15} {np.mean(rnd_steps):<15.0f}
{np.mean(rnd_rewards):.0f}')
      print(f'{"Greedy agent":<15} {np.mean(grd_steps):<15.0f}
{np.mean(grd_rewards):.0f}')
      print(f'{"PPO агент":<15} {np.mean(ag_steps):<15.0f}
{np.mean(ag_rewards):.0f}')

print(f"\nPPO vs без агента: {np.mean(ag_steps)/np.mean(no_steps):.2f}x")

```

```

print(f'PPO vs random:    {np.mean(ag_steps)/np.mean(rnd_steps):.2f}x")
print(f'PPO vs greedy:    {np.mean(ag_steps)/np.mean(grd_steps):.2f}x")

worst_no = np.argmin(no_steps)
print(f"\n=== КРИТИЧНИЙ СЦЕНАРІЙ (епізод {worst_no+1}) ===")
print(f'Без агента: {no_steps[worst_no]} кроків")
print(f'Greedy:    {grd_steps[worst_no]} кроків")
print(f'PPO агент: {ag_steps[worst_no]} кроків")
ratio = ag_steps[worst_no] / max(no_steps[worst_no], 1)
print(f'Покращення PPO vs без агента: {ratio:.0f}x")

return {
    'no_agent': {'steps': no_steps, 'rewards': no_rewards},
    'random':   {'steps': rnd_steps, 'rewards': rnd_rewards},
    'greedy':   {'steps': grd_steps, 'rewards': grd_rewards},
    'ppo':     {'steps': ag_steps, 'rewards': ag_rewards},
}

if __name__ == "__main__":
    evaluate()

```

topology.py

```

import grid2op
import numpy as np
import matplotlib.pyplot as plt

```

```

import matplotlib.patches as mpatches
from lightsim2grid import LightSimBackend
from grid2op.Parameters import Parameters

# Координати вузлів IEEE 14-bus (вручну для гарного вигляду)
NODE_POS = {
    0: (0.0, 2.0),
    1: (1.5, 3.0),
    2: (3.0, 3.0),
    3: (3.0, 1.5),
    4: (1.5, 1.0),
    5: (5.0, 3.5),
    6: (6.0, 2.5),
    7: (6.5, 3.5),
    8: (6.0, 1.5),
    9: (5.5, 0.5),
    10: (4.5, 0.0),
    11: (5.0, 1.5),
    12: (6.5, 1.0),
    13: (7.5, 2.0),
}

def draw_grid(ax, obs, env, title="Стан мережі"):
    ax.clear()
    ax.set_facecolor('#1a1a2e')
    ax.set_title(title, fontsize=11, fontweight='bold', color='white', pad=10)

    # Малюємо лінії
    for line_id in range(env.n_line):

```



```

or_bus = env.line_or_to_subid[line_id]
ex_bus = env.line_ex_to_subid[line_id]
x1, y1 = NODE_POS[or_bus]
x2, y2 = NODE_POS[ex_bus]

if not obs.line_status[line_id]:
    # Відключена лінія
    ax.plot([x1, x2], [y1, y2], color='#555555',
            linewidth=1, linestyle='--', alpha=0.5)
    continue

rho = obs.rho[line_id]
if rho < 0.7:
    color = '#4CAF50' # зелений — норма
    lw = 1.5
elif rho < 0.9:
    color = '#FFC107' # жовтий — увага
    lw = 2.5
else:
    color = '#F44336' # червоний — небезпека
    lw = 3.5

ax.plot([x1, x2], [y1, y2], color=color, linewidth=lw, alpha=0.9)

# Підпис завантаженості
mx, my = (x1+x2)/2, (y1+y2)/2
ax.text(mx, my, f'{rho*100:.0f}%', fontsize=6,
        color='white', ha='center', va='center',
        bbox=dict(boxstyle='round,pad=0.1', facecolor='#333333', alpha=0.7))

```

```

# Малюємо вузли
for node_id, (x, y) in NODE_POS.items():
    # Генератор?
    is_gen = node_id in env.gen_to_subid
    is_load = node_id in env.load_to_subid

    if is_gen:
        color = '#FFD700'
        size = 120
        marker = '*'
    elif is_load:
        color = '#64B5F6'
        size = 80
        marker = 'o'
    else:
        color = '#BBBBBB'
        size = 60
        marker = 'o'

    ax.scatter(x, y, c=color, s=size, marker=marker,
               zorder=5, edgecolors='white', linewidths=0.5)
    ax.text(x, y + 0.15, f'Bus {node_id}', fontsize=7,
            color='white', ha='center', va='bottom')

# Легенда
legend_elements = [
    mpatches.Patch(color='#4CAF50', label='< 70% (норма)'),
    mpatches.Patch(color='#FFC107', label='70-90% (увага)'),

```

```

mpatches.Patch(color='#F44336', label='> 90% (небезпека)'),
mpatches.Patch(color='#555555', label='Відключена'),
plt.Line2D([0], [0], marker='*', color='w', markerfacecolor='#FFD700',
            markersize=10, label='Генератор', linewidth=0),
plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='#64B5F6',
            markersize=8, label='Навантаження', linewidth=0),
]
ax.legend(handles=legend_elements, loc='lower left', fontsize=7,
          facecolor='#333333', labelcolor='white', framealpha=0.8)

ax.set_xlim(-0.5, 8.5)
ax.set_ylim(-0.5, 4.2)
ax.axis('off')

# ГОЛОВНА ДЕМОНСТРАЦІЯ
param = Parameters()
param.NO_OVERFLOW_DISCONNECTION = False
param.NB_TIMESTEP_OVERFLOW_ALLOWED = 2

env = grid2op.make("l2rpn_case14_sandbox",
                  backend=LightSimBackend(), param=param)

fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.patch.set_facecolor('#0f0f23')
fig.suptitle('Топологія енергомережі IEEE 14-bus',
            fontsize=14, fontweight='bold', color='white')

# Ліворуч — нормальний стан
obs = env.reset()

```

```

draw_grid(axes[0], obs, env, "Нормальний стан мережі")

# Праворуч — після відключення лінії
action = env.action_space({"set_line_status": [(17, -1)]}) # лінія 4_5_17
obs2, _, _, _ = env.step(action)
draw_grid(axes[1], obs2, env, "Після відключення лінії 4_5_17")

plt.tight_layout()
plt.savefig('topology.png', dpi=150, bbox_inches='tight',
           facecolor='#0f0f23')
plt.show()
print("Збережено: topology.png")

```

visualize.py

```

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

# =====
# 1. ГРАФІК НАВЧАННЯ (v2 дані)
# =====

timesteps_v2 = [
    2048, 4096, 6144, 8192, 10240, 12288, 14336, 16384, 18432, 20480,
    22528, 24576, 26624, 28672, 30720, 32768, 34816, 36864, 38912, 40960,
    43008, 45056, 47104, 49152, 51200, 53248, 55296, 57344, 59392, 61440,
    63488, 65536, 67584, 69632, 71680, 73728, 75776, 77824, 79872, 81920,

```

83968, 86016, 88064, 90112, 92160, 94208, 96256, 98304, 100352,
104448, 106496, 108544, 110592, 112640, 114688, 116736, 118784, 120832,
122880, 124928, 126976, 129024, 131072, 133120, 135168, 137216, 139264,
141312, 143360, 145408, 147456, 149504, 151552, 153600, 155648, 157696,
159744, 161792, 163840, 165888, 167936, 169984, 172032, 174080, 176128,
178176, 180224, 182272, 184320, 186368, 188416, 190464, 192512, 194560,
196608, 198656, 200704

]

ep_rew_v2 = [

180, 212, 250, 239, 303, 274, 280, 339, 434, 562,
532, 741, 1060, 1080, 1640, 1860, 2220, 2800, 3470, 3920,
4390, 4990, 5560, 6350, 7110, 7530, 7890, 8280, 8390, 8860,
9690, 10100, 9920, 10100, 10400, 10900, 11400, 11500, 12000, 12400,
12100, 12600, 12700, 12800, 12800, 13000, 13500, 13200, 13300,
18000, 18100, 19000, 19300, 19800, 19800, 20000, 20500, 20900,
21700, 22400, 22400, 21200, 21500, 22200, 22200, 22900, 23100,
22500, 21400, 21600, 21600, 21100, 20900, 20500, 20400, 21000,
20400, 19400, 18200, 17500, 17500, 18000, 18100, 18800, 19100,
19200, 19500, 19600, 19700, 20300, 20300, 21000, 21300, 21600,
21900, 22300, 23000

]

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

fig.suptitle('РРО агент v2 — керування енергомережею IEEE 14-bus\п200 000 кроків
навчання',

fontsize=13, fontweight='bold')

Крива навчання

```

ax1 = axes[0]
ax1.plot(timesteps_v2, ep_rew_v2, color='#2196F3', linewidth=1.5, alpha=0.6,
label='ep_rew_mean')

window = 7
smoothed = np.convolve(ep_rew_v2, np.ones(window)/window, mode='valid')
smooth_x = timesteps_v2[window-1:]
ax1.plot(smooth_x, smoothed, color='#F44336', linewidth=2.5,
        linestyle='--', label='Згладжена (MA-7)')

ax1.set_xlabel('Кроки навчання (timesteps)')
ax1.set_ylabel('Середня винагорода за епізод')
ax1.set_title('Крива навчання')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.yaxis.set_major_formatter(plt.FuncFormatter(
    lambda x, p: f'{x/1000:.0f}k' if x >= 1000 else f'{x:.0f}'))

ax1.axvspan(0, 20000, alpha=0.08, color='red')
ax1.axvspan(20000, 100000, alpha=0.08, color='yellow')
ax1.axvspan(100000, 200704, alpha=0.08, color='green')
ax1.text(5000, 1000, 'Дослідження', fontsize=8, color='red', alpha=0.8)
ax1.text(45000, 1000, 'Навчання', fontsize=8, color='olive', alpha=0.8)
ax1.text(140000, 1000, 'Збіжність', fontsize=8, color='green', alpha=0.8)

# Пікова позначка
peak_idx = ep_rew_v2.index(max(ep_rew_v2))
ax1.annotate(f'Пік: {max(ep_rew_v2)/1000:.1f}k',
            xy=(timesteps_v2[peak_idx], max(ep_rew_v2)),

```

```

xytext=(timesteps_v2[peak_idx]-30000, max(ep_rew_v2)-2000),
arrowprops=dict(arrowstyle='->', color='black'),
fontsize=9)

# =====
# 2. ПОРІВНЯННЯ v2
# =====

ax2 = axes[1]

episodes = ['Еп. 1', 'Еп. 2', 'Еп. 3', 'Еп. 4', 'Еп. 5', 'Середнє']
no_agent = [807, 3001, 3, 804, 513, 1026]
with_agent_v2 = [1261, 677, 806, 806, 3114, 1333]

x = np.arange(len(episodes))
width = 0.35

bars1 = ax2.bar(x - width/2, no_agent, width, label='Без агента',
                color='#EF5350', alpha=0.85)
bars2 = ax2.bar(x + width/2, with_agent_v2, width, label='PPO агент v2',
                color='#42A5F5', alpha=0.85)

for bar in bars1:
    h = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2, h + 20,
             f'{h:.0f}', ha='center', va='bottom', fontsize=7)
for bar in bars2:
    h = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2, h + 20,
             f'{h:.0f}', ha='center', va='bottom', fontsize=7)

```

```

# Стрілка на епізод 3 — найдраматичніший випадок
ax2.annotate('3 → 806\n(268x!)',
             xy=(x[2] + width/2, 806),
             xytext=(x[2] + width/2 + 0.6, 1200),
             arrowprops=dict(arrowstyle='->', color='green'),
             fontsize=8, color='green', fontweight='bold')

ax2.set_xlabel('Епізод')
ax2.set_ylabel('Кроків виживання')
ax2.set_title('Порівняння: агент v2 vs без агента')
ax2.set_xticks(x)
ax2.set_xticklabels(episodes)
ax2.legend()
ax2.grid(True, alpha=0.3, axis='y')
ax2.get_xticklabels()[-1].set_fontweight('bold')

plt.tight_layout()
plt.savefig('learning_curve_v2.png', dpi=150, bbox_inches='tight')
plt.show()
print("Збережено: learning_curve_v2.png")

app.py

# app.py — інтерактивний дашборд керування енергомережею
import streamlit as st
import matplotlib.pyplot as plt
import numpy as np

```



```

import pandas as pd
from src.env_setup import make_grid_env, make_gym_env
from stable_baselines3 import PPO
import json
import os

st.set_page_config(
    page_title="RL Керування Енергомережею",
    page_icon="⚡",
    layout="wide"
)

NODE_POS = {
    0:(0.0,2.0), 1:(1.5,3.0), 2:(3.0,3.0), 3:(3.0,1.5),
    4:(1.5,1.0), 5:(5.0,3.5), 6:(6.0,2.5), 7:(6.5,3.5),
    8:(6.0,1.5), 9:(5.5,0.5), 10:(4.5,0.0), 11:(5.0,1.5),
    12:(6.5,1.0), 13:(7.5,2.0),
}

@st.cache_resource
def load_model():
    env = make_grid_env()
    gym_env = make_gym_env(env)
    model = PPO.load("models/ppo_grid2op_v3", env=gym_env)
    return model

def draw_topology(obs, env, title, color_title):
    fig, ax = plt.subplots(figsize=(7, 5))
    fig.patch.set_facecolor('#1a1a2e')

```

```

ax.set_facecolor('#1a1a2e')
for lid in range(env.n_line):
    ob = env.line_or_to_subid[lid]
    eb = env.line_ex_to_subid[lid]
    x1, y1 = NODE_POS[ob]
    x2, y2 = NODE_POS[eb]
    if not obs.line_status[lid]:
        ax.plot([x1,x2],[y1,y2], color='#555', lw=1, ls='--', alpha=0.5)
        continue
    r = obs.rho[lid]
    color = '#4CAF50' if r < 0.7 else '#FFC107' if r < 0.9 else '#F44336'
    lw = 1.5 if r < 0.7 else 2.5 if r < 0.9 else 3.5
    ax.plot([x1,x2],[y1,y2], color=color, lw=lw)
    ax.text((x1+x2)/2, (y1+y2)/2, f'{r*100:.0f}%',
            fontsize=6, color='white', ha='center', va='center',
            bbox=dict(boxstyle='round,pad=0.1', fc='#222', alpha=0.7))
for nid, (x, y) in NODE_POS.items():
    is_gen = nid in env.gen_to_subid
    ax.scatter(x, y,
               c='#FFD700' if is_gen else '#64B5F6',
               s=150 if is_gen else 80,
               marker='*' if is_gen else 'o',
               zorder=5, edgecolors='white', lw=0.5)
    ax.text(x, y+0.18, f'Bus {nid}', fontsize=7,
            color='white', ha='center')
active = obs.line_status & (obs.rho > 0)
if active.any():
    mr = np.max(obs.rho[active])
    sc = '#4CAF50' if mr < 0.7 else '#FFC107' if mr < 0.9 else '#F44336'

```

```

ax.text(0.02, 0.02, f'Макс: {mr*100:.1f}%',
        transform=ax.transAxes, fontsize=9, color=sc, fontweight='bold',
        bbox=dict(boxstyle='round', fc='#333', alpha=0.8))
ax.set_title(title, color=color_title, fontsize=11, fontweight='bold')
ax.set_xlim(-0.5, 8.5); ax.set_ylim(-0.5, 4.2)
ax.axis('off')
plt.tight_layout()
return fig

```

@st.cache_resource

def scan_scenarios():

```

import grid2op
from lightsim2grid import LightSimBackend
from grid2op.Parameters import Parameters
param = Parameters()
param.NO_OVERFLOW_DISCONNECTION = False
param.NB_TIMESTEP_OVERFLOW_ALLOWED = 2
env_scan = grid2op.make("l2rpn_case14_sandbox",
                        backend=LightSimBackend(), param=param)
results = {}
for sid in range(20):
    obs = env_scan.reset(options={"time serie id": sid})
    done = False
    steps = 0
    while not done and steps < 200:
        obs, _, done, _ = env_scan.step(env_scan.action_space({}))
        steps += 1
    if steps < 10:
        label = f'Сценарій {sid} — критичний ⚠ '

```

```

elif steps < 100:
    label = f"Сценарій {sid} — важкий"
else:
    label = f"Сценарій {sid} — стабільний"
results[sid] = label
return results

```

```
#
```

```

st.title(" ⚡ RL Керування Топологією Енергомережі")
st.markdown("**IEEE 14-bus** | PPO агент vs Greedy vs без агента")
st.divider()

```

```
with st.sidebar:
```

```
    st.header("⚙️ Налаштування")
```

```

with st.spinner("Завантаження сценаріїв..."):
    scenario_labels = scan_scenarios()

```

```

scenario = st.selectbox(
    "Сценарій",
    options=list(scenario_labels.keys()),
    format_func=lambda x: scenario_labels[x],
    index=3
)

```

```
st.divider()
```

```

st.subheader(" ⚡ Сценарій аварії")

attack_type = st.radio(
    "Тип аварії",
    ["Без аварії", "Одна лінія", "Каскадна аварія (N-1-1)",
    index=0
)

line_names = [f"Лінія {i}" for i in range(20)]

disabled_line = "(не відключати)"
disabled_lines_cascade = []

if attack_type == "Одна лінія":
    disabled_line = st.selectbox(
        "Вибери лінію для відключення",
        ["(не відключати)" + line_names
    )
elif attack_type == "Каскадна аварія (N-1-1)":
    st.markdown("*N-1-1: дві лінії відключаються одна за одною*")
    line1 = st.selectbox("Перша лінія (відключається одразу)", line_names, index=17)
    line2 = st.selectbox("Друга лінія (відключається через 3 кроки)", line_names,
index=4)
    disabled_lines_cascade = [int(line1.split(" ")[1]), int(line2.split(" ")[1])]

st.divider()
max_steps = st.slider("Кроків симуляції", 20, 200, 100, step=10)
run_button = st.button("▶ Запустити симуляцію", type="primary",
                        use_container_width=True)

```

```
tab1, tab2, tab3, tab4 = st.tabs(["Симуляція", "Порівняння сценаріїв", "Архітектура",  
"Ручне керування"])
```

```
with tab1:
```

```
    if run_button:
```

```
        model = load_model()
```

```
    with st.spinner("Симуляція..."):
```

```
        env_no = make_grid_env()
```

```
        env_ag = make_grid_env()
```

```
        env_grd = make_grid_env()
```

```
        gym_env = make_gym_env(env_ag)
```

```
    # — Визначаємо тип аварії —————
```

```
    manual_off = None
```

```
    if attack_type == "Одна лінія" and disabled_line != "(не відключати)":
```

```
        manual_off = int(disabled_line.split(" ")[1])
```

```
    # — БЕЗ АГЕНТА —————
```

```
    steps_no, rho_history_no = [], []
```

```
    obs_no = env_no.reset(options={"time serie id": scenario})
```

```
    done_no = False
```

```
    cascade_step_no = 0
```

```
    if attack_type == "Каскадна аварія (N-1-1)" and disabled_lines_cascade:
```

```
        obs_no, _, done_no, _ = env_no.step(
```

```
            env_no.action_space({"set_line_status": [(disabled_lines_cascade[0], -1)]})
```

```
        )
```

```

elif manual_off is not None:
    obs_no, _, done_no, _ = env_no.step(
        env_no.action_space({"set_line_status": [(manual_off, -1)]})
    )

for _ in range(max_steps):
    if done_no: break
    if (attack_type == "Каскадна аварія (N-1-1)"
        and disabled_lines_cascade
        and cascade_step_no == 3
        and obs_no.line_status[disabled_lines_cascade[1]]):
        obs_no, _, done_no, _ = env_no.step(
            env_no.action_space({"set_line_status": [(disabled_lines_cascade[1], -
1)]})
        )
    else:
        obs_no, _, done_no, _ = env_no.step(env_no.action_space({}))
        cascade_step_no += 1
        steps_no.append(len(steps_no))
        active = obs_no.rho[obs_no.line_status]
        rho_history_no.append(np.max(active) if len(active) > 0 else 0.0)

# — PPO АГЕНТ —————
steps_ag, rho_history_ag = [], []
actions_count_ag = 0
event_log = []
cascade_step_ag = 0

obs_gym, _ = gym_env.reset(options={"time serie id": scenario})

```

```

obs_ag = env_ag.current_obs
done_ag = False

if attack_type == "Каскадна аварія (N-1-1)" and disabled_lines_cascade:
    obs_ag, _, done_ag, _ = env_ag.step(
        env_ag.action_space({"set_line_status": [(disabled_lines_cascade[0], -1)]})
    )
elif manual_off is not None:
    obs_ag, _, done_ag, _ = env_ag.step(
        env_ag.action_space({"set_line_status": [(manual_off, -1)]})
    )

prev_status_ag = obs_ag.line_status.copy()

for _ in range(max_steps):
    if done_ag: break

    # Каскадне відключення другої лінії через 3 кроки
    if (attack_type == "Каскадна аварія (N-1-1)"
        and disabled_lines_cascade
        and cascade_step_ag == 3
        and obs_ag.line_status[disabled_lines_cascade[1]]):
        obs_ag, _, done_ag, _ = env_ag.step(
            env_ag.action_space({"set_line_status": [(disabled_lines_cascade[1], -
1)]})
        )
    # Синхронізуємо gym_env
    obs_gym, _, term, trunc, _ = gym_env.step(
        gym_env.action_space.sample() * 0

```



```

    )
    obs_ag = env_ag.current_obs
    done_ag = term or trunc
else:
    act, _ = model.predict(obs_gym, deterministic=True)
    obs_gym, _, term, trunc, _ = gym_env.step(act)
    obs_ag = env_ag.current_obs
    done_ag = term or trunc

cascade_step_ag += 1

changed = np.where(obs_ag.line_status != prev_status_ag)[0]
for line_id in changed:
    status = "☑ підключено" if obs_ag.line_status[line_id] else "✗
відключено"

    active = obs_ag.rho[obs_ag.line_status]
    max_rho = np.max(active) if len(active) > 0 else 0
    event_log.append({
        'крок': len(steps_ag),
        'лінія': f"Лінія {line_id}",
        'дія': status,
        'макс_rho': f"{max_rho*100:.1f}%"
    })
    actions_count_ag += 1
prev_status_ag = obs_ag.line_status.copy()
steps_ag.append(len(steps_ag))
active = obs_ag.rho[obs_ag.line_status]
rho_history_ag.append(np.max(active) if len(active) > 0 else 0.0)

```

```

# — GREEDY AGENT —————
steps_grd, rho_history_grd = [], []
actions_count_grd = 0
cascade_step_grd = 0

obs_grd = env_grd.reset(options={"time serie id": scenario})
done_grd = False

if attack_type == "Каскадна аварія (N-1-1)" and disabled_lines_cascade:
    obs_grd, _, done_grd, _ = env_grd.step(
        env_grd.action_space({"set_line_status": [(disabled_lines_cascade[0], -1)]})
    )
elif manual_off is not None:
    obs_grd, _, done_grd, _ = env_grd.step(
        env_grd.action_space({"set_line_status": [(manual_off, -1)]})
    )

prev_status_grd = obs_grd.line_status.copy()

for _ in range(max_steps):
    if done_grd: break

    if (attack_type == "Каскадна аварія (N-1-1)"
        and disabled_lines_cascade
        and cascade_step_grd == 3
        and obs_grd.line_status[disabled_lines_cascade[1]]):
        obs_grd, _, done_grd, _ = env_grd.step(
            env_grd.action_space({"set_line_status": [(disabled_lines_cascade[1], -
1)]]})

```

```

    )
else:
    rho = obs_grd.rho.copy()
    rho[~obs_grd.line_status] = 0
    if np.max(rho) > 0.9:
        line_off = int(np.argmax(rho))
        action_grd = env_grd.action_space(
            {"set_line_status": [(line_off, -1)]}
        )
    else:
        action_grd = env_grd.action_space({})
    obs_grd, _, done_grd, _ = env_grd.step(action_grd)

    cascade_step_grd += 1
    if not np.array_equal(obs_grd.line_status, prev_status_grd):
        actions_count_grd += 1
    prev_status_grd = obs_grd.line_status.copy()
    steps_grd.append(len(steps_grd))
    active = obs_grd.rho[obs_grd.line_status]
    rho_history_grd.append(np.max(active) if len(active) > 0 else 0.0)

# — МЕТРИКИ —————
survived_no = len(steps_no)
survived_grd = len(steps_grd)
survived_ag = len(steps_ag)

attack_label = {
    "Без аварії": "штатний режим",
    "Одна лінія": f"відключення {disabled_line}",

```

```

"Каскадна аварія (N-1-1)": f'каскад: Лінія {disabled_lines_cascade[0]} → Лінія
{disabled_lines_cascade[1]}" if disabled_lines_cascade else ""
}.get(attack_type, "")

```

```

st.subheader(f'Результати симуляції — {attack_label}')
col1, col2, col3, col4 = st.columns(4)
col1.metric("Без агента", f'{survived_no} кроків")
col2.metric("Greedy агент", f'{survived_grd} кроків",
            delta=f'{survived_grd - survived_no:+d} кроків")
col3.metric("PPO агент", f'{survived_ag} кроків",
            delta=f'{survived_ag - survived_no:+d} кроків")
col4.metric("PPO vs Greedy",
            f'{survived_ag/max(survived_grd,1):.1f}x",
            delta="PPO кращий" if survived_ag > survived_grd else "однаково")

```

```

st.subheader("Детальні метрики")
m1, m2, m3, m4 = st.columns(4)
avg_rho_no = np.mean(rho_history_no) if rho_history_no else 0
avg_rho_grd = np.mean(rho_history_grd) if rho_history_grd else 0
avg_rho_ag = np.mean(rho_history_ag) if rho_history_ag else 0
m1.metric("Сер. завантаженість (без агента)", f'{avg_rho_no*100:.1f}%")
m2.metric("Сер. завантаженість (Greedy)",
          f'{avg_rho_grd*100:.1f}%"',
          delta=f' {(avg_rho_grd-avg_rho_no)*100:.1f}% (нижче = краще)',
          delta_color="inverse")
m3.metric("Сер. завантаженість (PPO)",
          f'{avg_rho_ag*100:.1f}%"',
          delta=f' {(avg_rho_ag-avg_rho_no)*100:.1f}% (нижче = краще)',
          delta_color="inverse")

```

```

m4.metric("Перемикань PPO / Greedy",
          f'{actions_count_ag} / {actions_count_grd}')

st.divider()

# — ЖУРНАЛ ПОДІЙ —————
st.subheader("Журнал дій PPO агента")
if event_log:
    df_log = pd.DataFrame(event_log)
    df_log.columns = ['Крок', 'Лінія', 'Дія', 'Макс. завантаженість']
    st.dataframe(df_log, use_container_width=True, hide_index=True)
    st.caption(
        f'Всього дій: {len(event_log)} | "
        f'Підключень: {sum(1 for e in event_log if '☑' in e['дія'])} | "
        f'Відключень: {sum(1 for e in event_log if '✗' in e['дія'])}"
    )
else:
    st.info("Агент не виконував перемикань — мережа була стабільною.")

st.divider()

# — ГРАФІК ЗАВАНТАЖЕНОСТІ —————
st.subheader("Максимальна завантаженість ліній")
fig_rho, ax_rho = plt.subplots(figsize=(10, 3))
fig_rho.patch.set_facecolor('#1a1a2e')
ax_rho.set_facecolor('#1a1a2e')
if rho_history_no:
    ax_rho.plot(rho_history_no, color='#EF5350', lw=2, label='Без агента')
if rho_history_ag:

```

```

    ax_rho.plot(rho_history_ag, color='#42A5F5', lw=2, label='PPO агент')
if rho_history_grd:
    ax_rho.plot(rho_history_grd, color='#FF9800', lw=2,
                label='Greedy агент', linestyle='-.')
ax_rho.axhline(y=1.0, color='#FF5722', lw=1.5, ls='--', alpha=0.7,
                label='Межа перевантаження (100%)')
ax_rho.axhline(y=0.9, color='#FFC107', lw=1, ls=':', alpha=0.5)
ax_rho.set_xlabel('Крок', color='white')
ax_rho.set_ylabel('Завантаженість', color='white')
ax_rho.tick_params(colors='white')
ax_rho.legend(facecolor='#333', labelcolor='white')
ax_rho.grid(True, alpha=0.2)
for sp in ax_rho.spines.values(): sp.set_color('#444')
plt.tight_layout()
st.pyplot(fig_rho)

```

```

st.divider()

```

— ТОПОЛОГІЯ —

```

st.subheader("Топологія мережі (фінальний стан)")
col_no, col_grd, col_ag = st.columns(3)
with col_no:
    fig_no = draw_topology(obs_no, env_no, "Без агента", '#EF5350')
    if done_no:
        st.error(" ⚡ БЛЕКАУТ!")
    st.pyplot(fig_no)
with col_grd:
    fig_grd = draw_topology(obs_grd, env_grd, "Greedy агент", '#FF9800')
    if done_grd:

```

```

        st.error(" ⚡ БЛЕКАУТ!")
    else:
        st.info(f" 🌀 Greedy: {survived_grd} кроків")
    st.pyplot(fig_grd)
with col_ag:
    fig_ag = draw_topology(obs_ag, env_ag, "PPO агент", '#4CAF50')
    if survived_ag > survived_no:
        st.success(f" ✅ PPO: {survived_ag} кроків")
    elif survived_ag == survived_no:
        st.info(f" ⓘ PPO: {survived_ag} кроків")
    else:
        st.warning(f" ⚠️ PPO: {survived_ag} кроків")
    st.pyplot(fig_ag)

st.divider()

# — КАРТА РИЗИКІВ —————
st.subheader("Карта ризиків вузлів")
node_risk = np.zeros(14)
node_risk_count = np.zeros(14)
for lid in range(env_ag.n_line):
    or_bus = env_ag.line_or_to_subid[lid]
    ex_bus = env_ag.line_ex_to_subid[lid]
    r = obs_ag.rho[lid] if obs_ag.line_status[lid] else 0
    node_risk[or_bus] += r
    node_risk[ex_bus] += r
    node_risk_count[or_bus] += 1
    node_risk_count[ex_bus] += 1

```

```

node_risk_norm = np.divide(
    node_risk, node_risk_count,
    where=node_risk_count > 0
)
fig_risk, ax_risk = plt.subplots(figsize=(10, 6))
fig_risk.patch.set_facecolor('#1a1a2e')
ax_risk.set_facecolor('#1a1a2e')
for lid in range(env_ag.n_line):
    or_bus = env_ag.line_or_to_subid[lid]
    ex_bus = env_ag.line_ex_to_subid[lid]
    x1, y1 = NODE_POS[or_bus]
    x2, y2 = NODE_POS[ex_bus]
    r = obs_ag.rho[lid] if obs_ag.line_status[lid] else 0
    color = '#4CAF50' if r < 0.7 else '#FFC107' if r < 0.9 else '#F44336'
    ax_risk.plot([x1,x2],[y1,y2], color=color, lw=1.5, alpha=0.5)
cmap = plt.cm.RdYlGn_r
for nid, (x, y) in NODE_POS.items():
    risk = node_risk_norm[nid]
    color = cmap(min(risk, 1.0))
    size = 200 + risk * 800
    ax_risk.scatter(x, y, c=[color], s=size,
                    zorder=5, edgecolors='white', lw=1.5, alpha=0.9)
    ax_risk.text(x, y, f'Bus {nid}\n{risk*100:.0f}%',
                 fontsize=7, color='white',
                 ha='center', va='center', fontweight='bold')
sm = plt.cm.ScalarMappable(cmap=cmap,
                            norm=plt.Normalize(vmin=0, vmax=100))
sm.set_array([])
cbar = plt.colorbar(sm, ax=ax_risk, shrink=0.6, pad=0.02)

```



```

cbar.set_label('Ризик (%)', color='white', fontsize=9)
cbar.ax.yaxis.set_tick_params(color='white')
plt.setp(cbar.ax.yaxis.get_ticklabels(), color='white')
ax_risk.set_title('Карта ризиків вузлів',
                  color='white', fontsize=11, fontweight='bold')
ax_risk.set_xlim(-0.8, 9.0); ax_risk.set_ylim(-0.8, 4.5)
ax_risk.axis('off')
plt.tight_layout()
st.pyplot(fig_risk)

top3 = np.argsort(node_risk_norm)[::-1][:3]
col_r1, col_r2, col_r3 = st.columns(3)
for i, (col, nid) in enumerate(zip([col_r1, col_r2, col_r3], top3)):
    risk_pct = node_risk_norm[nid] * 100
    is_gen = nid in env_ag.gen_to_subid
    node_type = " ⚡ Генератор" if is_gen else " ⚙ Навантаження"
    col.metric(f"#{i+1} Найризикованіший: Bus {nid}",
              f"{risk_pct:.1f}% ризик",
              delta=node_type, delta_color="off")

else:
    st.info("👉 Вибери сценарій у бічній панелі та натисни **Запустити симуляцію**")
    st.image("outputs/topology.png", caption="Приклад топології IEEE 14-bus")

with tab2:
    st.subheader("Порівняння всіх 20 сценаріїв")

    cache_file = "outputs/benchmark_results.json"

```

```

col_btn1, col_btn2 = st.columns(2)
with col_btn1:
    load_cache = st.button("Завантажити збережені результати",
                           type="primary", use_container_width=True)
with col_btn2:
    run_fresh = st.button("Перерахувати заново (~3 хв)",
                          use_container_width=True)

if load_cache and os.path.exists(cache_file):
    with open(cache_file, "r", encoding="utf-8") as f:
        rows = json.load(f)
    st.success("Результати завантажено миттєво!")
    show_results = True

elif run_fresh:
    with st.spinner("Тестування всіх сценаріїв (~3 хвилини)..."):
        rows = []
        progress = st.progress(0)
        for sid in range(20):
            env_base = make_grid_env()
            obs = env_base.reset(options={"time serie id": sid})
            done, steps_no = False, 0
            while not done and steps_no < 300:
                obs, _, done, _ = env_base.step(env_base.action_space({}))
                steps_no += 1

            env_ag = make_grid_env()
            gym_ag = make_gym_env(env_ag)

```

```

model_s = PPO.load("models/ppo_grid2op_v3", env=gym_ag)
obs_gym, _ = gym_ag.reset(options={"time serie id": sid})
done, steps_ag = False, 0
while not done and steps_ag < 300:
    act, _ = model_s.predict(obs_gym, deterministic=True)
    obs_gym, _, term, trunc, _ = gym_ag.step(act)
    done = term or trunc
    steps_ag += 1

env_grd = make_grid_env()
obs_grd = env_grd.reset(options={"time serie id": sid})
done, steps_grd = False, 0
while not done and steps_grd < 300:
    rho = obs_grd.rho.copy()
    rho[~obs_grd.line_status] = 0
    if np.max(rho) > 0.9:
        act_grd = env_grd.action_space(
            {"set_line_status": [(int(np.argmax(rho)), -1)]}
        )
    else:
        act_grd = env_grd.action_space({})
    obs_grd, _, done, _ = env_grd.step(act_grd)
    steps_grd += 1

label = scenario_labels[sid].split(" — ")[1]
rows.append({
    "scenario_id": sid,
    "type": label,
    "no_agent": steps_no,

```

```

        "greedy": steps_grd,
        "ppo": steps_ag,
        "ppo_vs_no": round(steps_ag/max(steps_no,1), 2),
        "ppo_vs_greedy": round(steps_ag/max(steps_grd,1), 2),
    })
    progress.progress((sid+1)/20)

    with open(cache_file, "w", encoding="utf-8") as f:
        json.dump(rows, f, ensure_ascii=False, indent=2)
        st.success("Результати збережені!")
    show_results = True

else:
    show_results = False
    if not os.path.exists(cache_file):
        st.warning("Збережених результатів немає. Натисни 'Перерахувати заново'")
    else:
        st.info("Натисни 'Завантажити збережені результати' для миттєвого завантаження")

if show_results and rows:
    df = pd.DataFrame(rows)
    df.columns = ["Сценарій", "Тип", "Без агента",
                  "Greedy", "PPO агент",
                  "PPO vs без агента", "PPO vs Greedy"]
    st.dataframe(df, use_container_width=True, hide_index=True)

fig_all, ax_all = plt.subplots(figsize=(12, 5))
fig_all.patch.set_facecolor('#1a1a2e')

```

```

ax_all.set_facecolor('#1a1a2e')
x = np.arange(20)
w = 0.25
no_vals = [r["no_agent"] if isinstance(r, dict) else r[2] for r in rows]
grd_vals = [r["greedy"] if isinstance(r, dict) else r[3] for r in rows]
ppo_vals = [r["ppo"] if isinstance(r, dict) else r[4] for r in rows]
ax_all.bar(x-w, no_vals, w, label='Без агента', color='#EF5350', alpha=0.85)
ax_all.bar(x, grd_vals, w, label='Greedy', color='#FF9800', alpha=0.85)
ax_all.bar(x+w, ppo_vals, w, label='PPO агент', color='#42A5F5', alpha=0.85)
ax_all.set_xlabel('Сценарій', color='white')
ax_all.set_ylabel('Кроків виживання', color='white')
ax_all.set_title('Порівняння всіх 20 сценаріїв',
                color='white', fontsize=12)
ax_all.set_xticks(x)
ax_all.set_xticklabels([str(i) for i in range(20)])
ax_all.tick_params(colors='white')
ax_all.legend(facecolor='#333', labelcolor='white')
ax_all.grid(True, alpha=0.2, axis='y')
for sp in ax_all.spines.values(): sp.set_color('#444')
plt.tight_layout()
st.pyplot(fig_all)

avg_ppo = np.mean(ppo_vals)
avg_no = np.mean(no_vals)
avg_grd = np.mean(grd_vals)
c1, c2, c3 = st.columns(3)
c1.metric("Сер. кроків без агента", f"{avg_no:.0f}")
c2.metric("Сер. кроків Greedy", f"{avg_grd:.0f}")
c3.metric("Сер. кроків PPO", f"{avg_ppo:.0f}",

```

$\delta = f' + \{avg_ppo - avg_no:.0f\}$ vs без агента")

with tab3:

```
st.subheader("Архітектура системи")
```

```
col_1, col_r = st.columns(2)
```

with col_1:

```
st.markdown("### 🌀 Reward Function")
```

```
st.latex(r'R(t) = \alpha \cdot S(t) - \beta \cdot O(t) - \gamma \cdot L(t) + \delta \cdot B(t)')
```

```
reward_data = {  
    "Компонент": ["S(t)", "O(t)", "L(t)", "B(t)"],  
    "Назва": ["Стабільність", "Штраф перевантаження", "Штраф блекауту",  
    "Бонус з'єднаності"],  
    "Коефіцієнт": [" $\alpha = 1.0$ ", " $\beta = 2.0$ ", " $\gamma = 100.0$ ", " $\delta = 0.5$ "],  
    "Опис": [  
        "+1 за кожен крок без аварії",  
        "-2 × сума перевантажень > 90%",  
        "-100 при блекауті",  
        "+0.5 × частка активних ліній"  
    ]  
}
```

```
st.dataframe(reward_data, use_container_width=True, hide_index=True)
```

```
st.markdown("### ⚙️ Гіперпараметри PPO")
```

```
ppo_params = {  
    "Параметр": ["learning_rate", "n_steps", "batch_size", "n_epochs", "ent_coef",  
    "clip_range", "gamma"],
```

```

"Значення": ["3×10-4", "2048", "64", "10", "0.01", "0.2", "0.99"],
"Призначення": [
    "Швидкість навчання",
    "Кроків до оновлення",
    "Розмір міні-батчу",
    "Епох на оновлення",
    "Коефіцієнт ентропії",
    "Діапазон кліпування",
    "Дисконт майбутніх нагород"
]
}
st.dataframe(ppo_params, use_container_width=True, hide_index=True)

```

with col_r:

```

st.markdown("### ☒ Порівняння версій моделі")
versions_data = {
    "Версія": ["v1 (100k кроків)", "v2 (200k кроків)", "v3 (300k + власна reward)"],
    "Reward Function": [
        "L2RPNReward (стандартна)",
        "L2RPNReward (стандартна)",
        "CustomReward (власна)"
    ],
    "Сер. кроків": [192, 1162, 2052],
    "Критичний сценарій": ["~13x", "~518x", "1661x"],
}
st.dataframe(versions_data, use_container_width=True, hide_index=True)
st.success("☒ v3 з власною reward function показує найкращі результати")

```

```

st.markdown("### 🧠 Нейромережа PPO (MlpPolicy)")

st.markdown("""
| Компонент | Деталі |
|-----|-----|
| Тип | Actor-Critic (2 мережі) |
| Шари | 2 приховані × 64 нейрони |
| Активація | tanh |
| Вхід | 114 параметрів |
| Вихід Actor | 101 дія (softmax) |
| Вихід Critic | 1 значення V(s) |
| Параметрів | ~30 000 |
""")

with tab4:
    st.subheader("Ручне керування мережею")
    st.markdown("Керуй мережею самостійно і порівняй результат з PPO агентом")

    col_ctrl_l, col_ctrl_r = st.columns([1, 2])

    with col_ctrl_l:
        st.markdown("### ⚙️ Налаштування")

        manual_scenario = st.selectbox(
            "Сценарій",
            options=list(scenario_labels.keys()),
            format_func=lambda x: scenario_labels[x],
            index=3,
            key="manual_scenario"
        )

```



```

manual_steps = st.slider(
    "Кроків симуляції", 10, 100, 30,
    key="manual_steps"
)

st.markdown("### 🛑 Вибери активні лінії")
st.markdown("*Зніми галочку щоб відключити лінію*")

# Ініціалізуємо стан ліній
if "line_states" not in st.session_state:
    st.session_state.line_states = [True] * 20

line_cols = st.columns(2)
for i in range(20):
    col_idx = i % 2
    with line_cols[col_idx]:
        st.session_state.line_states[i] = st.checkbox(
            f"Лінія {i}",
            value=st.session_state.line_states[i],
            key=f"line_{i}"
        )

run_manual = st.button(
    "▶ Запустити ручну симуляцію",
    type="primary",
    use_container_width=True,
    key="run_manual"
)

```

```

with col_ctrl_r:
    if run_manual:
        with st.spinner("Симуляція..."):
            # — РУЧНЕ КЕРУВАННЯ —————
            env_man = make_grid_env()
            env_ppo = make_grid_env()
            gym_man = make_gym_env(env_ppo)
            model_man = load_model()

            obs_man = env_man.reset(
                options={"time serie id": manual_scenario}
            )
            done_man = False
            steps_man = 0
            rho_man = []

            # Застосовуємо вибрані відключення
            for lid, active in enumerate(st.session_state.line_states):
                if not active:
                    obs_man, _, done_man, _ = env_man.step(
                        env_man.action_space(
                            {"set_line_status": [(lid, -1)]}
                        )
                    )
                    if done_man:
                        break

            # Симулюємо ручний режим
            while not done_man and steps_man < manual_steps:

```

```

obs_man, _, done_man, _ = env_man.step(
    env_man.action_space({})
)
steps_man += 1
active_lines = obs_man.rho[obs_man.line_status]
rho_man.append(
    np.max(active_lines) if len(active_lines) > 0 else 0.0
)

# — PPO на тому самому сценарії —————
obs_ppo, _ = gym_man.reset(
    options={"time serie id": manual_scenario}
)
obs_ppo_g2op = env_ppo.current_obs
done_ppo = False
steps_ppo = 0
rho_ppo = []

while not done_ppo and steps_ppo < manual_steps:
    act, _ = model_man.predict(obs_ppo, deterministic=True)
    obs_ppo, _, term, trunc, _ = gym_man.step(act)
    obs_ppo_g2op = env_ppo.current_obs
    done_ppo = term or trunc
    steps_ppo += 1
    active_lines = obs_ppo_g2op.rho[obs_ppo_g2op.line_status]
    rho_ppo.append(
        np.max(active_lines) if len(active_lines) > 0 else 0.0
    )

```

```
# — РЕЗУЛЬТАТИ —————
st.markdown("### Результат")
r1, r2, r3 = st.columns(3)
r1.metric("Твій результат", f'{steps_man} кроків")
r2.metric("PPO агент", f'{steps_ppo} кроків")

if steps_ppo > steps_man:
    diff = steps_ppo - steps_man
    r3.metric("Переможець", "PPO",
              delta=f'+{diff} кроків")
elif steps_man > steps_ppo:
    diff = steps_man - steps_ppo
    r3.metric("Переможець", "Ти",
              delta=f'+{diff} кроків")
else:
    r3.metric("Переможець", "🤝 Нічия", delta="0 кроків")
```

```
# — ГРАФІК —————
st.markdown("### 📊 Завантаженість ліній")
fig_man, ax_man = plt.subplots(figsize=(8, 3))
fig_man.patch.set_facecolor('#1a1a2e')
ax_man.set_facecolor('#1a1a2e')

if rho_man:
    ax_man.plot(rho_man, color='#EF5350', lw=2,
                 label='Твоє керування')
if rho_ppo:
    ax_man.plot(rho_ppo, color='#42A5F5', lw=2,
                 label='PPO агент')
```

```

ax_man.axhline(y=1.0, color='#FF5722', lw=1.5,
               ls='--', alpha=0.7, label='Межа (100%)')
ax_man.set_xlabel('Крок', color='white')
ax_man.set_ylabel('Завантаженість', color='white')
ax_man.tick_params(colors='white')
ax_man.legend(facecolor='#333', labelcolor='white')
ax_man.grid(True, alpha=0.2)
for sp in ax_man.spines.values():
    sp.set_color('#444')
plt.tight_layout()
st.pyplot(fig_man)

```

— ТОПОЛОГІЯ —————

```

st.markdown("### Топологія")
col_m1, col_m2 = st.columns(2)

with col_m1:
    fig_m1 = draw_topology(
        obs_man, env_man,
        "Твоє керування", '#EF5350'
    )
    if done_man:
        st.error("БЛЕКАУТ!")
    else:
        st.info(f"Ти: {steps_man} кроків")
    st.pyplot(fig_m1)

with col_m2:

```

```

fig_m2 = draw_topology(
    obs_ppo_g2op, env_ppo,
    "PPO агент", '#4CAF50'
)
if done_ppo:
    st.warning("PPO не зміг утримати")
else:
    st.success(f"PPO: {steps_ppo} кроків")
    st.pyplot(fig_m2)

else:
    st.info("☹️ Вибери які лінії відключити і натисни **Запустити**")
    st.markdown("""
**Як користуватись:**
1. Вибери сценарій навантаження
2. Зніми галочки з ліній які хочеш відключити
3. Натисни "Запустити ручну симуляцію"
4. Порівняй свій результат з PPO агентом

**Підказка:** спробуй відключити Лінію 17 і подивись
що станеться — це найнавантаженіша лінія в мережі!
""")

```

test_reward.py

"""

Unit-тести для функції винагороди CustomReward

Запуск: python -m pytest test_reward.py -v

```

"""

import pytest
from unittest.mock import MagicMock
import numpy as np

# -----
# Мок-реалізація CustomReward (незалежна від Grid2Op)
# Відтворює логіку з reward.py для ізольованого тестування
# -----

class CustomRewardMock:
    """Спрощена копія логіки CustomReward для тестування."""

    ALPHA = 1.0    # стабільність
    BETA = 2.0     # штраф перевантаження
    GAMMA = 100.0  # штраф блекауту
    DELTA = 0.5    # бонус з'єднаності
    OVERLOAD_THRESHOLD = 0.9

    def compute(self, rho: np.ndarray, is_blackout: bool, n_active_lines: int, n_total_lines:
int) -> float:
        """
        rho          — масив завантаженості ліній (0.0 – 1.0+)
        is_blackout   — True якщо стався блекаут
        n_active_lines — кількість активних ліній
        n_total_lines — загальна кількість ліній
        """

        if is_blackout:

```

```

        return -self.GAMMA

    S = self.ALPHA * 1.0
        O = self.BETA * float(np.sum(np.maximum(rho -
self.OVERLOAD_THRESHOLD, 0.0)))
    B = self.DELTA * (n_active_lines / n_total_lines)

    return S - O + B

reward_fn = CustomRewardMock()

# -----
# Тести
# -----

class TestCustomReward:

    def test_no_overload_no_blackout(self):
        """Нормальна робота мережі — очікується позитивна винагорода."""
        rho = np.array([0.5, 0.6, 0.4, 0.7, 0.3]) # всі < 0.9
        result = reward_fn.compute(rho, is_blackout=False, n_active_lines=5,
n_total_lines=5)
        assert result > 0, "При нормальній роботі винагорода має бути позитивною"

    def test_blackout_gives_large_penalty(self):
        """Блекаут — найбільший штраф, незалежно від стану ліній."""
        rho = np.array([0.1, 0.1, 0.1])
        result = reward_fn.compute(rho, is_blackout=True, n_active_lines=0,
n_total_lines=3)

```



```

assert result == -100.0, f"Штраф за блекаут має бути -100, отримано {result}"

def test_overload_reduces_reward(self):
    """Перевантажені лінії зменшують винагороду."""
    rho_normal = np.array([0.5, 0.5, 0.5])
    rho_overloaded = np.array([1.1, 1.2, 0.5]) # дві лінії перевантажені
    r_normal = reward_fn.compute(rho_normal, False, 3, 3)
    r_overloaded = reward_fn.compute(rho_overloaded, False, 3, 3)
    assert r_overloaded < r_normal, "Перевантаження має зменшувати винагороду"

def test_overload_threshold_boundary(self):
    """Лінія рівно на межі 90% — штрафу ще немає."""
    rho = np.array([0.9])
    result = reward_fn.compute(rho, is_blackout=False, n_active_lines=1,
n_total_lines=1)
    # O = 2.0 * max(0.9 - 0.9, 0) = 0
    assert result > 0, "Завантаженість 90% не має давати штраф"

def test_overload_above_threshold(self):
    """Лінія вище 90% — штраф нараховується."""
    rho_border = np.array([0.9])
    rho_over = np.array([1.0])
    r_border = reward_fn.compute(rho_border, False, 1, 1)
    r_over = reward_fn.compute(rho_over, False, 1, 1)
    assert r_over < r_border, "Завантаженість >90% має штрафуватись"

def test_connectivity_bonus(self):
    """Більше активних ліній — вища винагорода."""
    rho = np.array([0.5] * 20)

```

```

r_full = reward_fn.compute(rho, False, n_active_lines=20, n_total_lines=20)
r_partial = reward_fn.compute(rho, False, n_active_lines=10, n_total_lines=20)
assert r_full > r_partial, "Більше активних ліній має збільшувати винагороду"

def test_blackout_overrides_normal_state(self):
    """Блекаут дає -100 навіть якщо лінії не перевантажені."""
    rho = np.zeros(20) # всі лінії ненавантажені
    result = reward_fn.compute(rho, is_blackout=True, n_active_lines=20,
n_total_lines=20)
    assert result == -100.0

def test_reward_components_sum(self):
    """Перевірка що формула  $R = S - O + B$  правильно рахується вручну."""
    rho = np.array([1.0, 0.5]) # одна лінія перевантажена на 0.1
    #  $S = 1.0$ 
    #  $O = 2.0 * (1.0 - 0.9) = 0.2$ 
    #  $B = 0.5 * (2/2) = 0.5$ 
    #  $R = 1.0 - 0.2 + 0.5 = 1.3$ 
    result = reward_fn.compute(rho, False, n_active_lines=2, n_total_lines=2)
    assert abs(result - 1.3) < 1e-9, f"Очікувалось 1.3, отримано {result}"

```



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці

ГЛИБОКЕ НАВЧАННЯ З ПІДКРІПЛЕННЯМ ДЛЯ КЕРУВАННЯ ТОПОЛОГІЄЮ ЕНЕРГОМЕРЕЖ ПРИ РАПТОВИХ ВТРАТАХ ВУЗЛІВ ГЕНЕРАЦІЇ

виконала: студентка групи ТВ-23 Бунда Ольга Миколаївна

керівник: доцент, к.т.н Стативка Юрій Іванович

2026

Актуальність теми

Проблема: Каскадні аварії в енергомережах → блекауту → економічні збитки та загроза безпеці

Значення для ІПЗ: Розробка інтелектуальних систем керування складними динамічними середовищами на основі RL — актуальний напрям прикладного ML

Мета: Розробити програмну систему керування топологією мережі IEEE 14-bus на основі PPO, що запобігає блекаутам при аварійних відключеннях



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

2/13

Постановка задачі

- **Формулювання:** Керування топологією IEEE 14-bus формалізовано як марківський процес прийняття рішень $MDP = (S, A, P, R, \gamma)$
- **Простір станів:** вектор 114 параметрів
- **Простір дій:** 101 дискретна дія
- **Завдання:**
 1. Дослідити існуючі підходи
 2. Розробити математичну модель (MDP)
 3. Реалізувати власну функцію винагороди
 4. Навчити PPO агента на Grid2Op
 5. Провести порівняльне тестування
 6. Розробити веб-застосунок Streamlit

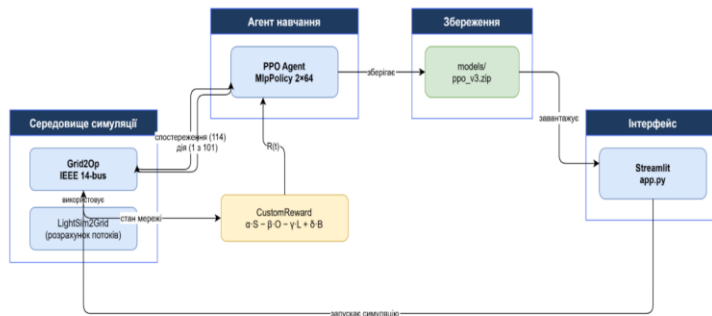


Аналіз предметної області

Метод	Швидкість	Якість рішення	Адаптивність	Складність реалізації
Жадібний алгоритм	Висока	Низька	Низька	Низька
MILP оптимізація	Низька	Висока	Низька	Висока
Random agent	Висока	Дуже низька	Відсутня	Дуже низька
PPO (навчання з підкріпленням)	Висока	Висока	Висока	Середня



Проектування системи



Функціональні вимоги:

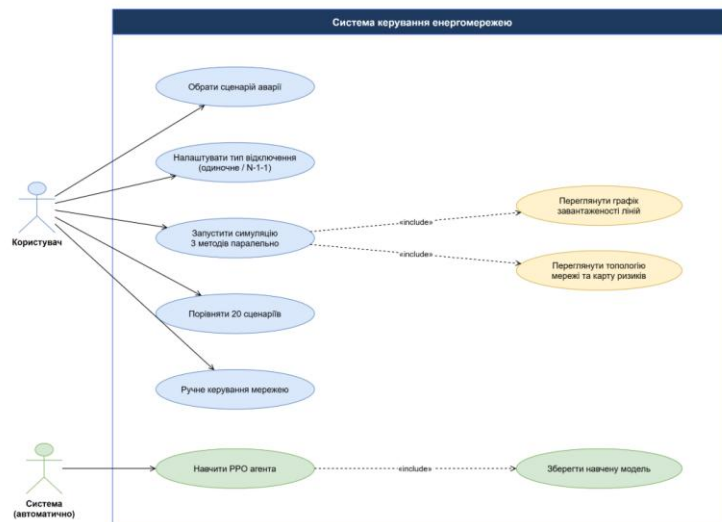
- Симуляція аварійних сценаріїв (одиначних і каскадних N-1-1)
- Навчання PPO агента з власною reward function
- Візуалізація топології та карта ризиків
- Порівняння з класичними алгоритмами
- Режим ручного керування



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

5/13

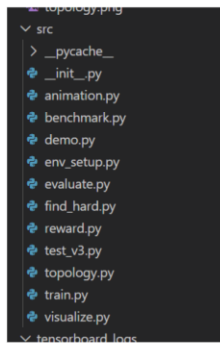
Проектування системи



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

6/13

Реалізація системи



Компонент	Технологія
Симулятор	Grid2Op + LightSim2Grid
RL алгоритм	PPO (Stable-Baselines3)
Веб-інтерфейс	Streamlit
Візуалізація	Matplotlib
Мова	Python 3.13



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

7/13

Реалізація: функція винагороди

$$R(t) = \alpha \cdot S(t) - \beta \cdot O(t) - \gamma \cdot L(t) + \delta \cdot B(t)$$

Компонент	Назва	Коефіцієнт	Опис
S(t)	Стабільність	$\alpha = 1.0$	+1 за кожен крок без аварії
O(t)	Штраф перевантаження	$\beta = 2.0$	-2 * сума перевантажень > 90%
L(t)	Штраф блекауту	$\gamma = 100.0$	-100 при блеауті
B(t)	Бонус з'єднаності	$\delta = 0.5$	+0.5 * частка активних ліній

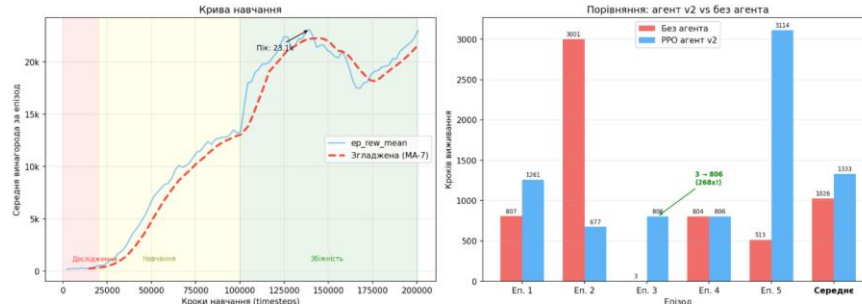


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

8/13

Реалізація системи

PPO агент v2 — керування енергомережею IEEE 14-bus
200 000 кроків навчання



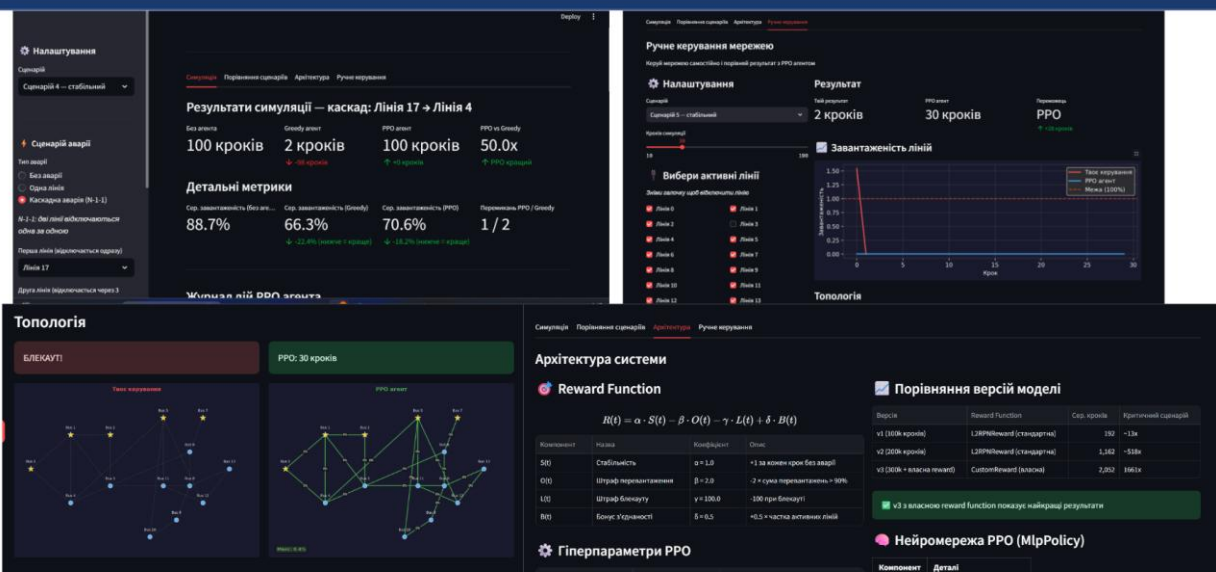
v1	L2RPNReward	100 000	192	~13x
v2	L2RPNReward	200 000	1162	~518x
v3	CustomReward	300 000	2052	~1661x



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

9/13

Реалізація: інтерфейс користувача



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

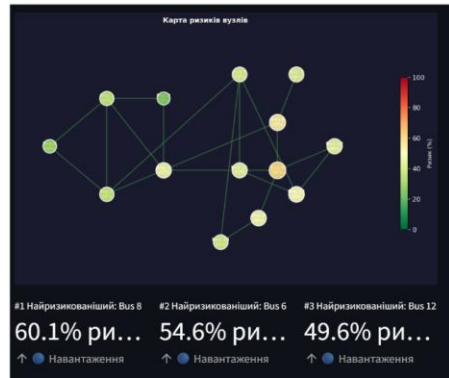
10/13

Тестування системи

```
PS C:\diplom> python -m pytest test_reward.py -v
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.
6.0 -- C:\Users\bonde\AppData\Local\Programs\Python\Pytho
n313\python.exe
cachedir: .pytest_cache
rootdir: C:\diplom
plugins: anyio-4.13.0, typeguard-4.5.1
collected 8 items

test_reward.py::TestCustomReward::test_no_overload_no_bla
ckout PASSED [ 12%]
test_reward.py::TestCustomReward::test_blackout_gives_lar
ge_penalty PASSED [ 25%]
test_reward.py::TestCustomReward::test_overload_reduces_r
eward PASSED [ 37%]
test_reward.py::TestCustomReward::test_overload_threshold
_boundary PASSED [ 50%]
test_reward.py::TestCustomReward::test_overload_above_thr
eshold PASSED [ 62%]
test_reward.py::TestCustomReward::test_connectivity_bonus
PASSED [ 75%]
test_reward.py::TestCustomReward::test_blackout_overrides
_normal_state PASSED [ 87%]
test_reward.py::TestCustomReward::test_reward_components_s
um PASSED [100%]

===== 8 passed in 1.37s =====
PS C:\diplom>
```



Порівняння всіх 20 сценаріїв

Завантажити збережені результати | Перерахувати заново (~3 хв)

Результати завантажено миттєво!

Сценарій	Тип	Без агента	Greedy	PPO агент	PPO vs Без агента	PPO vs Greedy
0	стабільний	300	300	300	1	1
1	стабільний	300	2	300	1	150
2	стабільний	300	300	300	1	1
3	критичний ⚠	3	1	3	1	3
4	стабільний	300	300	300	1	1
5	стабільний	300	300	300	1	1
6	стабільний	300	2	300	1	150
7	стабільний	300	300	300	1	1
8	стабільний	300	300	300	1	1



Висновки

- Досліджено існуючі підходи — обґрунтовано вибір PPO
- Розроблено математичну модель у вигляді MDP ($S=114$, $A=101$)
- Реалізовано власну функцію винагороди CustomReward
- Навчено PPO агента — 2 052 кроки виживання (v3)
- Проведено порівняльне тестування на 20 сценаріях
- Розроблено веб-застосунок Streamlit з 4 вкладками





Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

National Technical University of Ukraine
"Igor Sikorsky Kyiv Polytechnic Institute"